Master of Science in Informatics at Grenoble
Master Informatique
Specialization Data Science

Application of AI techniques for a smart execution of Domain Specific Languages

Marian Aldescu

June 21, 2021

Research project performed at VASCO team, LIG

Under the supervision of:
Akram Idani
and
Dominique Vaufreydaz

Defended before a jury composed of:
Head of the jury: Massih-Reza Amini
Examiner: Franck Iutzeler
External Expert: Michael Leuschel

June 2021

Abstract

Executable Domain Specific Languages (DSLs) have a great potential to reduce the development time of a system, by creating a model of it, before actually starting the implementation. Indeed, an executable DSL would not only exhibit the system's structure, but it will also emulate its expected behavior. The objective of this work is to explore how AI techniques can be useful while designing executable DSLs. Our contribution is applied to the Meeduse approach whose aim is to build and run correct DSLs thanks to the application of a formal method. In Meeduse the semantics of a DSL are written in the B language, which allows formal analysis and verification; and they are executed using ProB, an animator and model-checker of the B-Method. The problem addressed in this research is that the execution of a DSL is mainly done by interactive animation without any guidance: the user simply chooses the operations to execute and the tool animates them. To ensure a more convenient animation, we introduce within the tool a mechanism that allows it to learn and reproduce automated execution scenarios by itself. The task was naturally formalized as a Markov Decision Process, and the approach to solve it is entirely based on reinforcement learning algorithms, mainly due to the fact that they do not require a dataset to be trained on. An agent running on such methods will learn the goal to be achieved during the execution of the DSL by means of a reward function, which is itself designed in the B language. The advantage of the reward function, as proposed in this work, is that it is not hard-coded such as often done by existing works, but specified using domain-specific properties. Finally, we apply our proposal to the Self Driving Cab case-study, in order to lead the various experiments and have validation insights. The validation of the approach was successful, in the end highlighting the importance of investing effort in designing a good reward function to speed-up learning.

Acknowledgement

I would like to express my sincere gratitude to my supervisors, Mr. Akram Idani and Mr. Dominique Vaufreydaz, for their great responsiveness to my requests and their helpful comments in reviewing this report.

I also want to thank my office colleague, Mr. Asfand Yar, for his useful advice throughout my internship and for always encouraging me.

Résumé

Les Langages Exécutables Spécifiques au Domaine (DSL) ont un grand potentiel pour réduire le temps de développement d'un système, en créant un modèle de ce dernier, avant de commencer réellement la mise en œuvre. En effet, un DSL exécutable présenterait non seulement la structure du système, mais il simulerait également son le comportement attendu. L'objectif de ce travail est d'explorer comment les techniques d'IA peuvent être utiles lors de la conception de DSL exécutables. Notre travail est appliqué à l'approche Meeduse dont le but est de construire et d'exécuter des DSL corrects grâce à l'application d'une méthode formelle. Dans Meeduse, la sémantique d'un DSL est écrite en langage B, ce qui permet une analyse et une vérification formelle ; et ils sont exécutés à l'aide de ProB, un animateur et model-checker de la B-Method. Le problème abordé dans cette recherche est que l'exécution d'un DSL se fait principalement par animation interactive sans aucun guidage : l'utilisateur choisit simplement les opérations à exécuter et l'outil les anime. Pour assurer une animation plus pratique, nous introduisons au sein de l'outil un mécanisme qui lui permet d'apprendre et de reproduire par lui-même des scénarios d'exécution automatisés. La tâche a naturellement été formalisée en tant que processus décisionnel de Markov, et l'approche pour la résoudre est entièrement basée sur des algorithmes d'apprentissage par renforcement, principalement en raison du fait qu'ils ne nécessitent pas de jeu de données pour s'entraîner. Un agent fonctionnant sur de telles méthodes apprendra le but à atteindre lors de l'exécution du DSL au moyen d'une fonction de récompense, elle-même conçue en langage B. L'avantage de la fonction de compensation, telle que proposée dans ce travail, est qu'elle n'est pas codée en dur comme c'est souvent le cas dans les travaux existants, mais définie à l'aide de propriétés spécifiques au domaine. Enfin, nous appliquons notre proposition à l'étude de cas Self Driving Cab, afin de mener les différentes expérimentations et d'avoir des idées de validation. La validation de l'approche a été couronnée de succès, mettant finalement en évidence l'importance d'investir des efforts dans la conception d'une bonne fonction de compensation pour accélérer l'apprentissage.

Contents

Abstract i

Acknowledgement ii

Résumé iii

1 Introduction 1
- 1.1 Context . . . 1
- 1.2 Contribution . . . 2
- 1.3 Outline . . . 3

2 Related Work 5
- 2.1 Safe Reinforcement Learning via Formal Methods . . . 6
- 2.2 Reinforcement Learning techniques to repair models . . . 7
- 2.3 Model Transformations using Neural Networks . . . 8

3 Background 11
- 3.1 Executable Domain Specific Languages . . . 11
  - 3.1.1 Static versus Dynamic Semantics of DSLs . . . 11
  - 3.1.2 Model vs Meta-model . . . 12
  - 3.1.3 Meeduse approach for Executable DSLs . . . 13
  - 3.1.4 ProB approach for Model checking and animation . . . 14
- 3.2 Reinforcement Learning . . . 15
  - 3.2.1 General concepts . . . 15
  - 3.2.2 Finite Markov Decision Processes . . . 17
  - 3.2.3 Solving an MDP . . . 18
  - 3.2.4 Q-learning . . . 19
  - 3.2.5 SARSA . . . 21
  - 3.2.6 Q(λ) . . . 22
  - 3.2.7 Importance of the reward function and initial Q-values . . . 24

4 Experimental Methods 27
- 4.1 Case Study - Self Driving Cab . . . 27
  - 4.1.1 Meta-model . . . 28

4.1.2 Static semantics . . . 29
4.1.3 Dynamic semantics . . . 30
4.1.4 Reward function . . . 31
4.2 Results and Discussion . . . 32
4.2.1 Performance measurement . . . 33
4.2.2 Comparison of reward functions . . . 35
4.2.3 Initial Q-values impact on convergence . . . 37
4.2.4 Discovered versus precomputed state space . . . 37

5 A Deep Learning approach to express the Q-table **41**

6 Conclusion and Future Work **45**

A Appendix **47**

Bibliography **51**

— 1 —
Introduction

1.1 Context

Increasing software complexity could be seen as a long-standing problem that occurred in the early years of software development, almost five decades ago, and probably present also in those to come, if not properly addressed, especially in large development organizations. In order to deal with this issue, diverse techniques were incrementally adopted, like enumerating beforehand the requirements that should be satisfied by the system [8], how to design the architecture [37], the actual implementation, tests formulation. Entire technologies were proposed, a good representative of them is Computer Aided Software Engineering (CASE) [4], which aimed to accompany the developers throughout the entire software life cycle. Despite its advantages, like: producing systems with longer operational life and closer to user's needs, good documentation and applications that requires less technical support, some important inconveniences, such as difficulty to customize and use, the high cost to build and maintain new systems, lead organisations to reevaluate it and reorient to other solutions. As described in [16], we have real reasons to hope that Model Driven Engineering (MDE) [13] methodologies will succeed to overcome some of the previous limitations. The main contribution of MDE is to introduce new levels of abstraction across the software life cycle, in order to decrease developer's endeavor, simplify and formalize the whole process of designing and implementing a system.

Under the MDE umbrella, Executable Domain Specific Languages (DSLs) enable building a model of the system, then analyzing its behaviour for inconsistencies, before jumping to develop complex applications. We will refer to this activity as *debugging*. To a large extent, it can vary from solving small ambiguities or errors to completing tedious data acquisition and analysis tasks. Notable factors in the ability to debug an issue are the debugging skills of the developer, together with the complexity of the system, the programming language used and the software debugging tools.

Usually, debugging is a cyclic process, during which, after the DSL is created or refined, the modeler will execute it and navigate through its state space to check if the requirements are satisfied, and if not, the steps are repeated. By *if the requirements are satisfied* we mean that, starting possibly from any state, for reaching some target states, the sequence of needed actions to be performed on the model is the desired one.

Significant drawbacks of such an approach is that in the debugging loop, it is very likely that the state space will change, thus, the same sequence of actions is no longer valid, or the state space could be considerably large, therefore, a manual exploration could be impractical.

A brute force inspection of all the possible action paths is out of discussion, thus there is a need for an intelligent recommendation of which actions to perform for specific states. With this in mind, we would like to explore the usefulness of machine learning techniques, with the aim of predicting these actions, towards a specific goal defined by the programmer.

An essential tool exploited in this contribution is Meeduse [18], a Language workbench with interactive debugging facilities, based on B formal method [1]. The strength of Meeduse is its ability to apply DSLs in safety-critical systems because it allows taking advantage of automated reasoning tools.

1.2 Contribution

After investigating the state of the art, we realized that debugging using domain specific languages together with machine learning were not explored, thus our work is a first attempt in this direction. Taking into account that each action heads to a particular next state and it has associated with it an unknown probability that it will lead to the final target state, we can situate our task in the Markov Decision Process (MDP) framework. Solving MDP problems can be done using reinforcement learning (RL) techniques [36], in which an agent gains knowledge from scratch by performing actions on the environment and being rewarded or penalized. In this study we focus on two representatives, Q-learning and SARSA, both of them can be endowed with *eligibility traces*, obtaining Q(λ) and SARSA(λ). Another method to solve MDPs that we considered is Monte Carlo. The reason for selecting them is that they are representative of several important concepts in RL, like: on/off-policy learning, time-difference methods, and trace eligibility. These techniques come together with good aspects and disadvantages, as explained later, one of the limitations being its restriction to not large DSLs, due to the fact that the information learned consists of numerical values, associated to each state, stored in a structure called Q-table. We call *large DSLs* those with a very high number of possible valid states. When dealing with large state spaces, it can increase and reach impractical dimensions. To overcome such a drawback, the Q-table can be replaced with a neural network, which can be seen as an universal function approximator, obtaining the solution called Deep Q-learning. Also this method comes in turn with challenges, e.g. unguaranteed convergence, how to represent the states and actions, so that the neural to extract useful information from them.

In this work, we pay special attention to safety-critical systems, in other words, it is not sufficient to check the correctness of a DSL through verification and validation techniques, but it needs to be proved by formal means [19], such as model checking, static analysis or program proofs. If we refer to the RL methods applied to safety-critical use cases, we will find them in the literature under the name of Assured Reinforcement Learning (ARL) [23], which restricts the agent to perform a subset of all available actions, that ensures valid solutions with respect to the expected constraints.

With the aim to provide safe behaviour to a DSL, we apply B-method to define its formal specification, to which the agent must comply. AtelierB [6] is used for proving the specification, and ProB [20] to check the model and animate its execution. These are contained in Meeduse, which additionally allows the translation of a meta-model expressed using Eclipse Modelling Framework (EMF) [34] into an equivalent formal B specification, in which an instance of the DSL is then injected.

We keep in mind that the current work is situated at the intersection of several fields: MDE, RL, and formal methods. Although they are different well established domains, in the proposed

solution we naturally bring together notions from each of them, to provide new insights into smart executing DSLs. The main idea that should accompany the reader throughout the paper can be summarized as: the execution of a DSL, whose static semantics are defined using MDE and dynamic semantics via formal methods, can be learned with RL techniques. By its nature, a DSL is designed to fulfill a certain purpose, e.g to reach some target states, an aspect easily expressed through the *reward function*, a fundamental concept in Reinforcement Learning. We show the utility of the proposed methods on a study case, Self Driving Cab, which shortly consists of learning a taxi how to reach a client and transport it to the destination, both of them being located on a well delimited map. In the context of RL, the taxi is an agent which gathers knowledge by interacting with elements on the map, whose behaviour is to maximize a score influenced by the quality of the chosen sequence of actions. The goals are, first, to reach the client and pick him up, then to transport him to the desired location, both of them directed through receiving a reward or a punishment for each action. The central idea of this problem, and of the current work in general, is to guarantee safety of the agent, therefore, it is not allowed to perform risky actions that may put him in danger. Choosing to work on this case-study is due to the fact that it is a concise and complete example, doable in the context of an internship, and also, the overall approach presented here can be applied for more complex problems. We can consider it a representative of all DSLs composed of pieces that are placed on a map and moved in order to achieve an objective, hence, as this informal definition is formulated, it can include many-real world tasks. Another research object, towards which our efforts are directed, is chess. Chess DSL exhibits a non-negligible challenge likely to occur when dealing with real-world systems, which is a possibly very large state space. This issue is usually addressed in the literature using neural networks, so we would like to extrapolate this solution to large DSLs. Even though this task is in progress, we will summarize our approach at the end.

1.3 Outline

The report is organised as follows:

- Chapter 2: Related work. In this chapter, we review several existing works and explain how our contribution is influenced by them.
- Chapter 3: Background. It provides the necessary theoretical background on Model Driven Engineering, Reinforcement Learning and a glimpse on B formal method.
- Chapter 4: Experimental Methods. Here, we propose, Self Driving Cab, a case study to validate our methods, present its static and dynamic semantics, together with several reward functions. In the end we analyze the results obtained after training the algorithms described in the background chapter.
- Chapter 5 A Deep Learning approach to express the Q-table. We present a more complicated study case, chess, but which is planned especially for future work.
- Chapter 6: Conclusion and Future Work. In this chapter we draw several conclusions and give directions for future contributions.

— 2 —
Related Work

Throughout this chapter, several existing methods from the literature that we consider relevant to our subject will be reviewed. To our knowledge, there are no contributions that explicitly investigate the smart execution of DSLs, but we will discuss instead some works mainly concentrated on safe reinforcement learning or which combine AI techniques with model driven engineering. In the field of formal methods, there are numerous efforts that apply AI methods to expand new ways of formally checking the models, but it is not the case of the current project. Our work is primarily concerned with automatizing the modelling endeavor and not with verification, although the latter is still employed to ensure the safety of taken actions.

Even though the idea behind this work is to take advantage of AI techniques, with the intention of endowing DSLs agents with some degree of autonomy when executed, we can also naturally change the perspective and situate it in AI world, and more precisely in Safe Reinforcement Learning (safe RL) context, which has several important contributions. Shortly, this sub-domain is characterized as being the process of searching for policies that maximize the expectation of the return, applied for tasks in which the system should satisfy some performance or/and safety constraints.

There are two main approaches within the safe RL field [12]: *Optimization Criterion* and *Exploration Process*. The first one consists of modifying the optimality criterion such that the learned policy provides a risk sensitive control, which can be adjusted to allow a certain amount of risk failure. As there still exists a probability of performing invalid actions, possibly leading to disastrous states [14], we did not retain them. The second category is more interesting, as, according to its philosophy, safe learning is driven either using external knowledge, or by employing a risk metric. The latter is not so desirable, considering that the risk metric is approximated during training, alongside policy learning. What we would like is to have a well established failure measurement from the beginning.

Exploration aided by external knowledge can be done in several ways. *Providing Prior Knowledge* is one of them, which consists of initializing the agent with information about the domain, such as a set of demonstrations provided by a teacher and used to train a regression agent [10]. Some other method of exploiting external knowledge is allowing the agent to ask for advice from a teacher. This technique is called *Ask for help* algorithms [7] and depend on a confidence parameter, which, for example, in the case of Q-learning, suggests a low confidence whenever the Q-values are close and vice-versa. Another solution to help the agent with external information is through teacher advice, whenever he feels needed. Compared to the previous one, this time, the agent is not any more in charge of deciding when to ask for help, but a human teacher can interfere with additional guidance. For example in [22], the interven-

tion of the teacher comes in a set of rules, and when one of them is satisfied, it is indicated weather a Q-value should have a small or a large value, in order to execute an action. Earlier presented methods focus on the safety of the agent, but do not provide formally proved critical properties. Fortunately, late research proposed some solutions that qualifies them to this regard, as in [23], which exploits Abstract MDPs (AMDP) to model the agent-environment interaction and probabilistic model checking to build policies that respect constraints expressed in probabilistic temporal logic.

The following part will cover three contributions that we considered pertinent to position our effort in the literature. The first one [11] combines formal methods and RL on Cyber-physical systems. The second paper [2] envisions the task of repairing broken models as a reinforcement learning problem. We will interpret this as a specific case of domain specific language. The third one [3] explores how models can be transformed using neural networks. After resuming each work, we explain what is the relation with our work and the eventual improvements.

2.1 Safe Reinforcement Learning via Formal Methods

In a recent work [11], reinforcement learning is accompanied by formal methods and applied to assure the safe operation of Cyber-physical systems. The solution is called Justified Speculative Control (JSC), which consists of transferring formally verified results to RL controllers. The computer checked proofs are injected at runtime in the RL controlled agent, which basically learns a non deterministic policy which is able to perform only safe actions. More precisely, JSC behaves as a monitoring instrument of the learning process and makes sure that transitions performed by the agent comply with a formally proved model of the environment, defined by a set of differential equations. This case is valid only if there are no inaccuracies in the modeled system, otherwise safety cannot be guaranteed.

Even if a good model of a system is accurate most of the time, the study raises awareness that even such a model could be incomplete, and even in this case, empirical evidence is brought to show that safe RL can benefit from formally verified results.

The safety verification is done via Differential Dynamic Logic, a first-order multi-modal logic to specify and prove properties of hybrid systems. An important part of the work is the possibility to check at runtime if the safety properties of a model are violated. This aspect is employed by a method called ModelPlex [25] in two ways: on the one hand it provides *control monitors*, which are boolean functions that check if a portion of a system is valid or not, and on the other hand it can check the accuracy of the entire model using *model monitors*. When learning, JSC algorithm (Algorithm 1) enables an agent to choose valid actions only if the model of the environment is accurate, or a random action otherwise. After one of these steps, the current state and underlying learning model are updated.

We pay attention to this study, because it establishes a broad intuition on how our work will look like, but with significant differences. The main focus of the paper falls more on the formal verification side and mainly applied on cyber-physical systems. On the other hand, we insist on the modelling side of DSLs, which provide safety properties by the way they are designed in our approach, so the effort to ensure them is considerably less. Second, we formally prove the behaviour of a DSL using B method, and the model is checked at runtime via ProB, while here, the verification is done using differential dynamic logic.

Algorithm 1 Justified Speculative Control Generic

```
1:  procedure LEARNING(init, (S,A,R,E), choose, update, done, CM, MM)
2:      prev ← curr ← init
3:      a0 ← NOP
4:      while notdone(curr) do
5:          if MM(prev, a0, curr) then
6:              u ← choose(a ∈ A | CM(a, curr))
7:          else
8:              u ← choose(A)
9:          prev ← curr
10:         curr ← E(u, prev)
11:         update(prev, u, curr)
```

2.2 Reinforcement Learning techniques to repair models

In MDE literature, applying AI techniques is still in the early stages and has a lot of potential. Although, at first glance, the task of repairing damaged models using RL methods [2] may not seem directly related to our work, it can be viewed, however, as a normal DSL, in which actions modify attributes of the model, hoping the resulted one has fewer errors. Also this work inspired the methodology used later in the report, in regard to the applicability of some classical reinforcement learning techniques on a MDE task. The paper is formulated in addition to a previous effort of the authors, exploring the same subject, but now comparing the performance of several methods.

The problem of repairing models can be expressed as a Markov Decision Process, in which states are represented by the errors of a model, the number of each category: classes, attributes, operations, references. Hence, at the beginning of an episode, the initial state is initialized with the set of elements previously mentioned. The available actions are a set of predefined operations that modifies the model: *delete, setName*, *setType*, etc. Ideally, applying an action will transit the current state into another one with less errors, so the purpose of an agent is to learn the shortest sequence of actions that reach a state with an empty set of errors (terminal state). The solutions for this task were found using well known reinforcement learning algorithms: Q-learning, SARSA, SARSA(λ), Q(λ). The reward function was designed as follows: every action leading to a state different than the final one is rewarded with 0, otherwise, the value is provided by an external tool that measures the quality of a model.

After training the algorithms, the obtained results can be seen in Table 2.1. Their performance was measured in terms of execution time until convergence and the improvement was measured setting Q-learning as reference. It seems that Q(λ) exceeded all the other methods, with an improvement of 3.53%, compared to Q-learning.

	Q-learning	Q(λ)	Monte Carlo	SARSA	SARSA(λ)
Total time(s)	2931.56	2828.07	3093.75	4720.08	3093.75
Improvement	-	3.53%	-3.8%	-61%	-5.53%

Table 2.1: Running time of RL algorithms compared with Q-learning

Even though the subject of repairing models is not of direct interest for us, the overall applied procedure, to compare different reinforcement learning methods, could help validate the approach proposed by us in the next chapters. To understand our contribution to this paper, the reader should step back and visualize it from a DSL perspective and not from a repairing damaged models view. We will realize that a model can be in fact formulated as a domain specific language. For example, according to our work, the actions can be defined as B specifications, instead of manually writing them in a general purpose programming language. In this context, we implicitly have the necessary actions to change a model, as they are normally used when converting a meta-model in B. Nonetheless, the reward function used when training the agent with reinforcement learning is basically hard-coded, considering it comes from an external provenance. As we envision our proposal, detailed in Chapter 4, this function can be instead designed as a B specification, without depending on external tools. In fact, the versatility of B allows designing multiple reward functions, hence they can be compared and to see which one is the most suitable. In this regard, imagination is the limit.

2.3 Model Transformations using Neural Networks

Another recent example of how MDE can benefit from the use of AI inspired methods can be found in [3], which will be the main principal subject of the next discussion. It addresses an important need in the domain, and this is model translation. The contribution occurred in response to the lack of solutions enabling non-expert users to automatically design transformation rules, without domain specific knowledge. Although there are methods to facilitate this process, they are usually tedious and error-prone. The proposed approach is based on neural networks (NN), and more specifically, on Long Short Term Memory (LSTM), a NN architecture designed to process sequential data. Since they have been successfully applied on different tasks of sequence-to-sequence translation [35], model transformation was envisioned to fall into this category, hence the choice for LSTM to learn transformation rules that map input models to target output models. During the experiments (described in the next paragraphs), simple LSTMs were proven not to be enough to produce good results, so a more complex NN architecture was tried, that is Encoder-Decoder endowed with attention. A key part of the work was to find a proper way to encode the input models in sequence of numbers, that conserves as much information as possible from the original format. This stage is usually required when training NNs and known as *pre-processing* or *vectorization*. First, each model is represented as a tree, where *objects* and *associations* are children of the root. We do not go more into details about the graph structure, the reader can refer to [3] for this. An example of such a tree can be seen in Figure 2.3.

One issue needed to be solved was the *dictionary problem*, which occurred because of the possibility to have any string for the name of attributes and variables. Overcoming it was done by changing the names to a restricted set of strings, so the NN will be able to recognize them for any input model. The overall structure of the network is: a step of pre-processing the input models and representing them as trees, which are converted into numerical format, followed by a LSTM layer (encoder), an attention layer, a second LSTM layer (decoder), then a step of sequence-to-tree, to obtain the graph model, and finally during post-processing, the names of objects are restored.

The performance evaluation is done as for classical machine learning classification tasks, using the accuracy of predicting the expected output model. Other aspects are measured, like

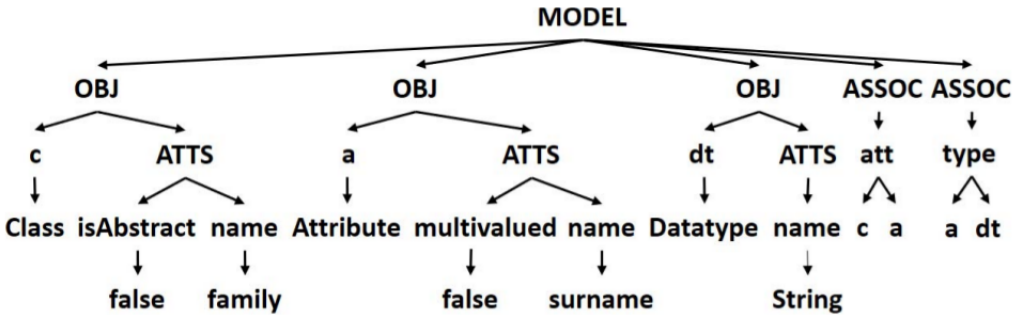

Figure 2.1: Representing a model as a tree.

assessing the variation of training time with respect to the dimension of the dataset and the size of the models, or the time the network needs to do predictions.

A pertinent discussion takes place at the end of the paper, in which some issues of NNs are analyzed, related to the task of model transformation. Some of them are: the need of large training datasets in order to exploit the full potential of NNs, the limitations to perform mathematical operations or to generalize well. Another drawback is the lack of acceptance of NNs due to their nature of *black-box*.

At first sight, the connection between the work just presented and our task may not be obvious. We should instead see an executable domain specific language as a model transformation problem. For example, let's consider an agent based on the semantics of a DSL. When it performs an action a_t on the state s_t which transits the environment in s_{t+1}, actually it takes place a transformation from a model at time t to a model at time $t+1$. In the end, the trace of executed actions of the agent will leave a trace of chained models. These being said, the encoder-decoder NN architecture earlier discussed could be replaced by a certain combination of successive actions to reach a target model from an initial one. The solved limitations would be all those related to neural networks, mentioned in the previous paragraph. As probably intuited already, the agent's training will be guided by means of a reward function designed by the modeler. An important advantage brought by our method of designing executable DSLs is that any reached state (target model) is ensured to be correct with respect to the formal B specification, an aspect hardly guaranteed by a NN. Even though in our experiments we do not approach the task of model transformation, we are optimistic about their applicability to it. Specific details on how to build such a DSL will be seen later. We are aware that we only scratch the surface with our hypothesis. However, it could be a key starting point for solutions to a large range of problems, since we propose a general method of formulating them as executable DSLs.

— 3 —
Background

This chapter introduces the key concepts needed to understand how executable DSLs work in general and how their execution can be directed to achieve specific goals using reinforcement learning. We show the detailed steps to build a DSL in the context of Model Driven Engineering accompanied by B-method, by means of Eclipse Modelling Framework and Meeduse. We will insist on the main components of a DSL, the *static* and *dynamic semantics*, and how its state space is animated and checked using ProB. Finally, we focus on the modelling side of a domain specific language and how its execution can be performed in a smart manner to achieve specific goals. This problem will be formulated as a Markov Decision Process and solved using well established reinforcement learning solutions in the domain, such as Q-learning and SARSA algorithms.

3.1 Executable Domain Specific Languages

A DSL is in fact a software language, but constructed for a certain domain [39]. As domain experts use a specific jargon specific to their field, in order to make the communication more efficient, it is the same for computer DSLs, whose instructions are designed to perform precise tasks. In MDE space, DSLs started to occupy an significant role, as they define a set of models that can be built in the domain, and so, tools like code generators or model translators benefit from that representation, exploiting it to obtain source code, formal specifications, animations (*animation* - related to the *Animator*, presented later).

3.1.1 Static versus Dynamic Semantics of DSLs

Building an executable DSL essentially means to define its semantics, with respect to the domain at hand, and they are composed of two main components: *static semantics* and *dynamic semantics*.

Static semantics, also called *abstract syntax*, describes how a DSL looks like, captures the modeling concepts, together with the attributes and the relationship between them. We name all these elements the structure of a DSL. In MDE field, the static syntax can be depicted by means of a *meta-model*, presented in the next subsection. Providing only the meta-model is not enough to have a complete semantic of the domain, because it is difficult to include DSL's behavior in it. There exist methods to translate the meta-model into a general purpose language (such Java) [17], allowing explicit programming of the behavior, and it is the same direction

we follow. An important mention is that we use B language [1], which is usually employed to develop formally provable safety critical systems. The translation of the meta-model in B is done by Meeduse tool [18], on which we insist later.

Dynamic semantics, known as operational or executable semantics, is in charge of specifying how a DSL must behave, in other words, makes it executable, by depicting the actions that can be performed on the subjacent model. Executing actions of a DSL and globally observing its behavior contribute to understanding if the effect is the expected one. The meaning of this is to better understand the possible causes of eventual faults in the model or inconsistencies between the model and the system. As for the abstract syntax, we use B to specify the dynamic semantics. Two concepts characterize B: *abstract machine* and *refinement*.

An abstract machine allows designing a system specification in B, which starts with simple mathematical constructs, like sets, constants, properties, all these elements being able to express data from the domain at hand: *user*, *name*, *attributes*, etc.. Another two parts define, on the one hand, the state of a model at a certain point in time, composed of *variables* and *invariant* and on the other, the operations: *initialization*, *operations*. The invariant is a central notion in the abstract machine, which defines the static properties of data inside a model, as a result of an "invariant" predicate. From this point, the consistency of a model is strictly related to the proof obligations to be met by the invariant.

After defining a first (possibly very abstract) model, details can be gradually introduced in it, obtaining a more concrete one. This process is called *refinement* and it basically defines the relationship between consecutive models. Finally, it is intended to reach a complete and executable specification and the purpose of using B is to obtain a proved model. The reason is to guarantee a correct behavior, which is at the same time the expected one by the designer. We see the system model as a running instance of the DSL, hence each executing step of the model should transfer it in a safe state, so it implies performing a valid transition. The rules of what is allowed and what not are expressed in B language.

3.1.2 Model vs Meta-model

The main motivation behind MDE [13] approach when developing software is that, before starting building the desired system (i.e. writing the code), a *model* should be created and then refined, until it is as close as possible to the real world use-case. A good model will have the following characteristics:

- abstraction - it has a good ratio of hiding the system's complexity and keeping enough information about the structure and its behavior;
- easy to understand;
- accurate with respect to the real life problem;
- predictive - easy to guess possible hidden properties, considerably;
- less expensive to conceive than the real system.

Another essential notion is the *meta-model*, which specifies the properties that should be contained in a valid model and at the same time, how they should be structured. A simplified analogy, to get a better grasp about these two notions, is when creating objects in an object

oriented language. The class definition is the meta-model, while its instance is the model. Furthermore, the meta-model itself can be seen as a model of a modelling language. In our case, the modeling language is called Meta Object Facility (MOF) and we use it in EMF.

EMF is a modelling framework with code generation facilities [34], based on which, applications with an underlying data model can be built. One of its important elements is EMF core (Ecore) (Figure 3.1), a generic meta-model that allows describing models by its instantiation. As Ecore is the matrix responsible for creating meta-models, to be entirely correct, we should call it a meta-meta-model. The concept of meta-model is crucial for us, as it is a fundamental part when defining executable DSLs as we will find out in the next section.

Before going further, we need to clarify the terminology regarding the concept of *model*, considering that it can occur in different domains. First, we will use it with the meaning of instance of a meta-model for MDE aspects, and we just saw what that means. A second connotation is related to Reinforcement Learning. It can be used for example in contexts like *model of an environment* or as *Markov Decision Process model*. The latter two will be described later.

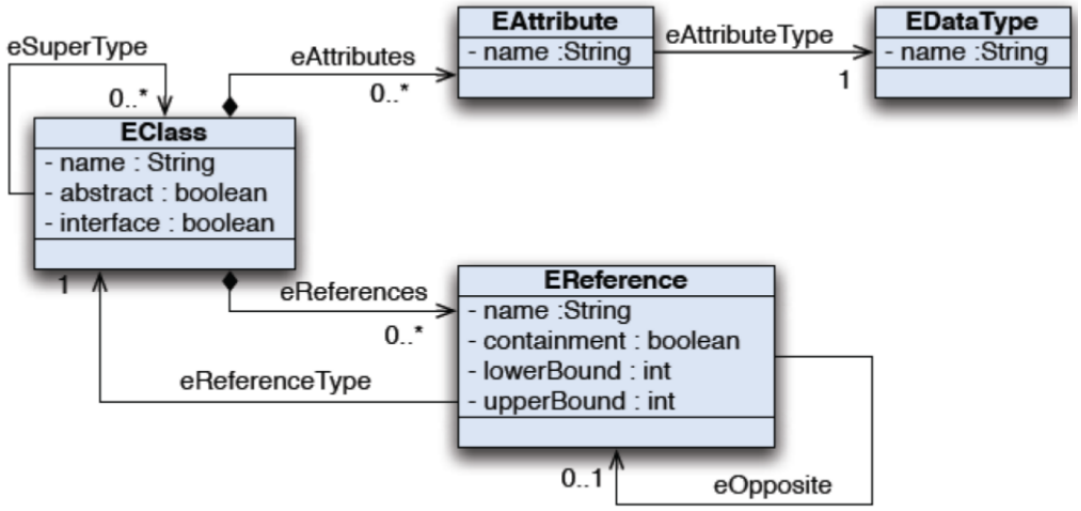

Figure 3.1: Ecore meta-model

3.1.3 Meeduse approach for Executable DSLs

Meeduse[1] allows model-driven engineering to exploit the advantages of formal methods for defining formal semantics of domain specific languages [18]. The tool connects, on the one hand, EMF [34], representative of MDE, which uses the Ecore meta-model to build the abstract syntax, on the other hand, B-method to obtain the B machine and define the abstract behavior to be proved and refined, and finally the runtime environment, determined by the dynamic semantics of the DSL. The architecture can be visualized in Figure 3.2. Meeduse is composed of four main components, each of them summarized next:

[1] http://vasco.imag.fr/tools/meeduse/

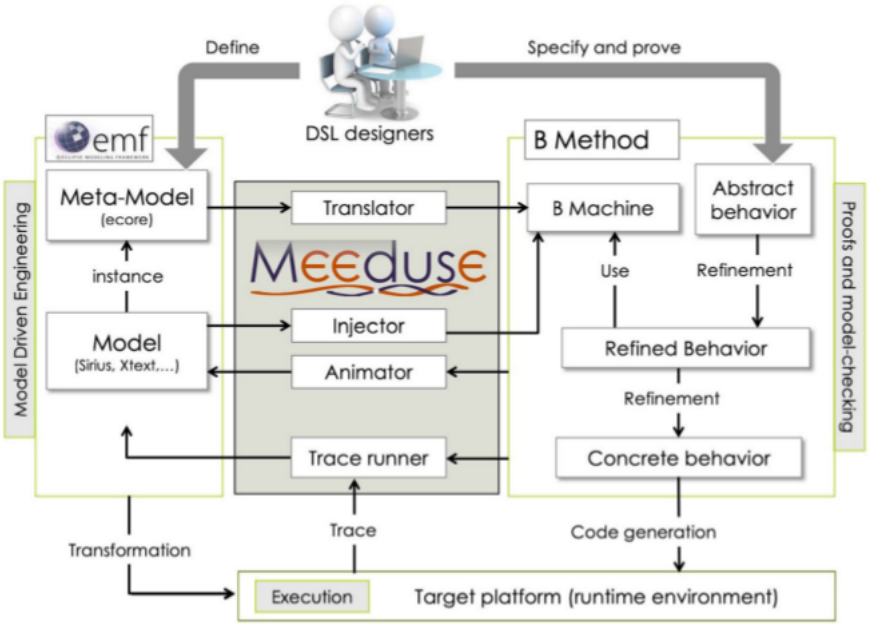

Figure 3.2: Meeduse architecture and interaction with EMF, B-method and execution of the system [18].

1. Translator: it is in charge of transforming an Ecore meta-model in its correspondent B specification, which can be manually enhanced by adding operational semantics and concrete behavior. After reaching a satisfying final specification, to demonstrate that the properties expressed in it are coherent and not contradictory, an automatic prover can be used, such as Atelier B[2] [33].

2. Injector: it is capable of taking an instance of the meta-model designed in EMF and *injects* it into the B specification of the meta-model, obtaining a B machine, that can be valuated and model checked over finite domains.

3. Animator: this component is based on the ProB model checker and animator [20] for B method. In Meeduse, the animation is done by calling ProB and applying B operations on the current state, so the model will transition into a new state. The latter is then acquired by the Animator through B variables valuations and will synchronize it in the EMF model.

4. Trace runner: it allows the execution of a sequence of actions that may come from an external source, for example an agent that runs the execution platform.

3.1.4 ProB approach for Model checking and animation

The B Method is applied with two main objectives: consistency and refinement checking, and we explained them when discussing about DSLs. The ProB tool [20] allows one to prove the consistency of a formal B specification via *model checking* [5]. For small finite sets it allows reaching all the states, and for larger models, it is able to explore and look for errors in a subset

[2]https://www.atelierb.eu/en/

of the state space. It is also capable of checking the consistency using *constraints*, by detecting a state that fulfills the invariant and from which, an invalid state can be reached after performing a single action.

Another important component of ProB is the *model animator*. As the name suggests, the user can execute step-by-step actions and visualize the resulting graph. Performing incremental transitions on a B-machine coming from an external source is very helpful in our case, because we can plug-in a program that interacts with it, observes the changes occurring in the model and exploits them. This implies receiving some sort of feedback for each action, and it is possible to specify it together with the B specification. This feedback is called a *reward*, but we define it later. When using transitions, elements like state IDs, and destinations are always transmitted, but some others are lazily retrieved only when needed, to increase the performance by avoiding to transmit and parse large strings. Even so, overhead will still exist, but fortunately, ProB has the option to exhaustively evaluate the state space and save the graph in a specific format (DOT). We want to see, after trying both possibilities, what is the time difference between them.

In the process of developing a model, after prototyping it, usually the designer wants to test how close it is to the real world problem. This requires testing, for example, if certain nodes from the model's state space can be reached and also if the sequence of actions taken is the expected one. Even though the DSL is proved to be formally verified, it still can be far away from the targeted model, namely, the correctness does not guarantee a good model. Furthermore, the task could be also thought of as a collaborative debugging process between a human modeler and an autonomous action recommender system. Even so, the job remains the same: reaching targeted states.

In Figure 3.3, originally found in [18], it can be seen the modelling environment of Meeduse, which calls ProB to check and animate the state space. The environment is composed of four views. The graphical view (1) shows a visual representation of the DSL. The output state view (2) shows the enumerated sets expressed in B. The last two views allow the model to be edited in two ways: using the properties view (3), based on the EMF plugin and finally, using the animation view (4) with B operations.

3.2 Reinforcement Learning

Now that we have a precise definition of what a DSLs is and how it works, we detail the necessity of applying *reinforcement learning*, before jumping to the theory behind it.

At this moment we are very close to RL, but there is a missing piece and that is: how to notify the agent that it performed a good or a bad action. It can be done by indicating from the beginning a rewarding strategy. The simplest one is to give positive rewards for the gratifying actions, and negative rewards for actions that either are really bad, either they are neutral and move away the agent from the optimal sequence of steps. On this idea, we can build other more complex rewarding procedures. We arrived to the point where RL naturally can be introduced.

3.2.1 General concepts

Reinforcement Learning means to learn how to map situations to actions, by maximizing a reward given from an external source [36]. The learner is not explicitly told what to do, but

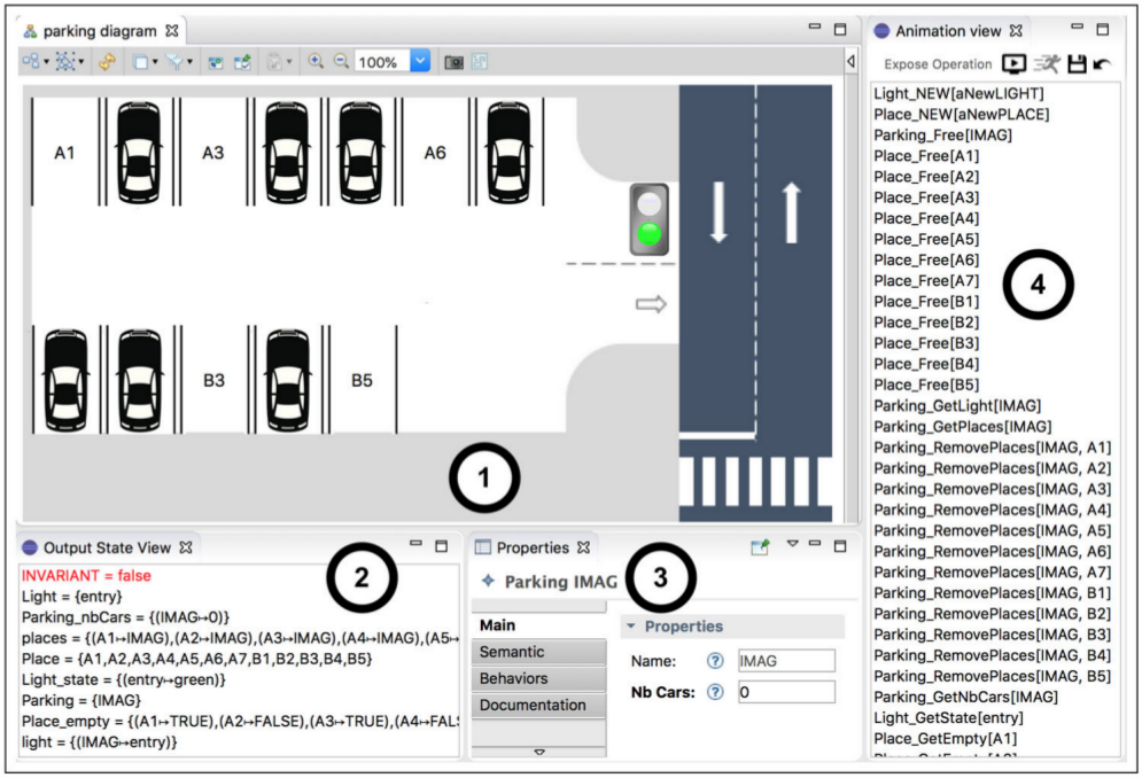

Figure 3.3: Screenshot of the modelling view of Meeduse using ProB [18].

it is indicated if the chosen action yields a punishment or a reward. Besides this *trial-and-error* aspect of RL, there is another distinctive feature, that is *delayed-reward*, meaning to learn which actions are very likely to lead to a significant future reward, even if on the short term they can cause a diminishment.

A big challenge of RL is to optimally address the trade-off between *exploration* and *exploitation*. Exploitation refers to the situations when the agent discovers that some actions give a small reward, so it may choose to perform them indefinitely. It should instead be let to a certain extent to execute short-term dummy steps, in order to be able to explore new regions that could reward it better. Usually, this kind of balance is employed by choosing random actions with a probability different than 0.

Specific RL terminology that we will use throughout the study is:

- *Policy*: It determines the behavior of the agent at a certain moment of time and overall describes the learned strategy. In practice, it can appear in diverse forms, like simple functions, lookup tables or even more complex logic like searching algorithms (Monte-Carlo, minimax, etc.).

- *Reward*: It can be seen as a feedback from the environment for each of the learner's actions and establishes what is the agent's purpose to approximate an optimal policy. Usually it is represented as a mapping between pairs (*state, action*) to a number.

- *Value function*: Compared to the reward, which indicates the immediate quality of an action, the value function contains information about what is preferable to do for the long term. Broadly speaking, the *value* of a state approximates the total expected reward

to receive in the future, by choosing an action that transits to it. In practice, the value function can be represented by any means that are able to approximate a function: lookup tables, neural networks, etc..

3.2.2 Finite Markov Decision Processes

The RL task can be mathematically formalised using finite Markov Decision Processes (finite MDPs), a framework for sequential decision problems. The primary conditions are: the learning agent is placed in an unknown environment, and initialised with a state S_t. At each time step t, the agent chooses an action from the available ones, based on the current policy, and executes it on the environment, and this will answer with a reward R_{t+1} and the next state S_{t+1}. This is summarized in Figure 3.5, from [36].

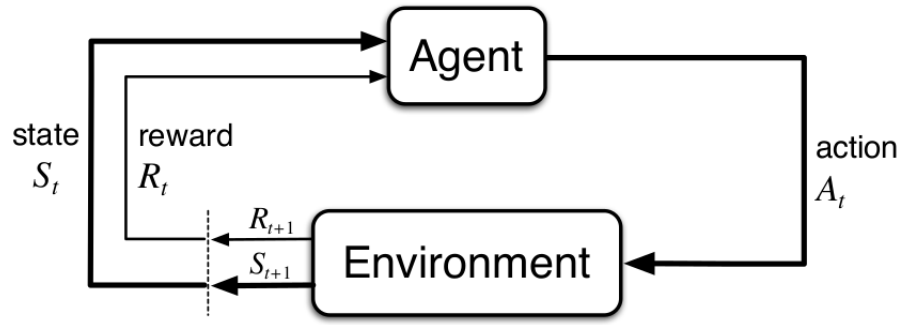

Figure 3.4: The interaction between an agent and the environment.

An MDP is defined by four elements:

- S is the set of all possible states.
- A is the set of all available actions, or if a state has a specific set of possible actions, we note it $A(s)$.
- $R(s,a)$ is the real valued reward function
- $P(s'|s,a)$ is a model that gives the probability of reaching a state s' from a state s by performing an action a.

MDPs are founded on a Markov Chain (MC), according to which, the system evolves in discrete time steps and respects the assumed *Markov Property*: the probability of transitioning to a next state S_{t+1} depends only on the action A_t chosen in the previous state S_t. In other words, the current state must contain information about all the previous actions chosen by the agent. A good analogy is a chess position: the board has information about all the moves performed by two players, and the number of moves or their order does not influence the next move to be chosen.

In MDP framework, an agent's goal is to look for choosing actions that maximize the *cumulative reward*, starting from some state S_0 (note that several states can be chosen as the initial one) and ending with a *terminal* state, that stops learning. This delimitation is called *episode*. Normally, the agent goes through multiple episodes, the number depending one the size of the problem. We also remember that it is not necessary for an action to produce a high return in

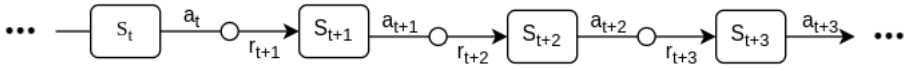

Figure 3.5: Markov Chain - discrete sequence of *(state,action)* pairs.

the short run, but we look for long term high rewards. These being said, the purpose can be redefined as to find the mapping between states and actions $\pi : S \to A$ that is as close as possible to the optimal policy π^*. We say *as close as possible* because, even though in theory we can reach the optimal policy, in practice it is not always achievable (e.g. when the state space is very large, so computationally impractical).

Another thing that needs to be clarified is how we reach, from a short term reward, to a long run return, expressed, as we described earlier, by the *value function*, which gives the possible future total return of choosing a state. We can formally express this as $V^\pi : S \to \mathbb{R}$ in Eq. 3.1:

$$V^\pi(s_t) = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{i=0}^{\infty} \gamma^i r_{t+i} \tag{3.1}$$

where r_t is the reward received at time t and $\gamma \in [0,1]$ is a parameter called *discount factor*. The discount factor controls the importance given by the agent to future rewards. When γ is close to 0, the learner will focus on immediate rewards, while if γ is close to 1, the agent will act with stoicism and aim for future rewards.

We can rewrite Eq. 3.1 in a way that shows better the relation between consecutive states, in terms of returns:

$$\begin{aligned} V^\pi(s_t) &= r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \cdots) \\ &= r_t + \gamma V^\pi(s_{t+1}) \end{aligned} \tag{3.2}$$

and it can be read as: the value of the current state depends on the immediate reward plus a discounted value of the next chosen state. This will be useful when discussing about Q-learning algorithm in the following section.

3.2.3 Solving an MDP

Solutions for MDPs can be found using different methods, so I will briefly present the most well known ones, as the goal of the work is not necessary to do an exhaustive comparison among them.

Dynamic Programming

The most straightforward ones are formulated as *dynamic programming* tasks, term introduced by Richard Bellman in the '50s, which designates a class of algorithms that decompose a problem in smaller subproblems, in a recursive manner. As authors states in [36], standard DP algorithms are not of great importance for RL in practice, due to high computational requirements and the need of a perfect model of the environment, but still, they provide significant theoretical foundations to understand other more complex approaches. In the context, two representative algorithms for DP are *Value Iteration* and *Policy Iteration*. Both of them relay on Bellman optimality equation:

$$V^{\pi^*}(s) = \max_{a \in A(s)} \{R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^{\pi^*}(s')\} \tag{3.3}$$

for any states s, $s' \in S$, action $a \in A(s)$, where $V^{\pi^*}(s)$ is the value of a state with respect to the unknown optimal policy π^*. Note that if we have n states, there are n non-linear equations, because of *max*. Value Iteration algorithm finds the optimal values by transforming Eq.3.3 into an update rule, so it computes the values in an iterative manner, until there are no changes between 2 consecutive steps. Then, the optimal policy is calculated using Eq.3.4.

$$\pi^*(s) = arg \max_{a \in A(s)} \sum_{s'} P(s'|s,a) V^{\pi^*}(s')\} \tag{3.4}$$

On the other hand, Policy Iteration computes iteratively the optimal policy directly in a loop together with the utility values. We will not go further into details about implementation, but the reader can reference to [36] for a more in-depth study. We will resume to say that both algorithms reach the optimal policy, but they are too computationally expensive, due to nested loops, therefore impractical for large MDPs.

Monte Carlo Methods

Also like TD techniques, an agent using a Monte Carlo (MC) method learns through interactions with the environment, but in a slightly different manner: it generates experience samples, and at the end, the utility of each state is updated by averaging the returns gathered during the exploration. An inconvenience of this solution is the requirement of training to be organized in episodes, in other words a terminal state needs to be reached before computing the state values. At the same time, completing an episode does not guarantee that the state space was fully explored, so the results can be affected by a high variance.

Temporal-Difference Learning

Some of the most important concepts in RL arose with the discovery of Temporal Difference (TD) Learning techniques. The name itself, *temporal difference*, suggests that the iterative updating steps come from the differences between consecutive transitions. They are inspired by Monte Carlo methods, because both of them can learn directly from raw occurrences of events, without requiring a full model of the environment. At the same time, TD uses concepts from DP, as the computing the outcome for a certain step is based on previously calculated results. The central idea of TD is that a state will gain value if the agent performs an action with a positive resulting outcome, and reduces the value otherwise. The feedback for TD methods can occur from only one transition and it is called TD(0), or from multiple transitions, so it becomes n-step TD(0) or TD(λ). TD(0) updating rule is expressed in Eq.3.5.

$$V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)] \tag{3.5}$$

TD approach in reinforcement learning is important because it establishes the foundation of some well known algorithms like Q-learning, SARSA and Q(λ).

3.2.4 Q-learning

Solutions for MDPs can be found by applying Q-learning, probably the most known and researched RL algorithm [36]. It is a representative of the temporal difference methods, but with a variation: instead of computing the values of each state, as for TD(0) for example, it does it iteratively for each pair $(state, action)$. All these values are stored in a tabular form, called

Q-table. Q-learning is a *model-free* technique, meaning that it does not require a model of the environment. Here (in comparison with MDE), a model represents a transition graph with all the states in which the agent can be found, with edges as actions and nodes as states. The Q-table is the approximation of a *quality* function, so if S is the set of states, A - the set of actions and R the set of possible rewards, the relationship between them is:

$$Q : S \times A \to \mathbb{R} \tag{3.6}$$

The Q-values are updated according to Bellman's equation (Eq.3.7), but applied on (s,a) pairs. After taking and action a_t, from a state s, the maximum future reward of s - the $Q(s_t,a_t)$ value - is modified with a quantity that depends on the actual received reward r plus the maximum future reward of the next state s_{t+1}, discounted by a factor $\gamma \in [0,1]$. If γ close or equal to 0, the agent will only focus on short-run rewards, otherwise, if $\gamma = 1$, it will concentrate too much on long-term rewards. This mechanism allows the propagation of a high reward throughout the sequence of transitions, from the end to the beginning, only for the actions that lead to a high outcome. Actions that only lead the agent farther away from the high return states will suffer a decrease in their associated Q-value. $\alpha \in [0,1]$ is the learning rate and defines how fast Q-values change over time. In practice, usual α values are close to 0, like 0.1 or 0.01.

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha[r_{t+1} + \gamma \max_{a' \in A(s_{t+1})} Q(s_{t+1},a') - Q(s_t,a_t)] \tag{3.7}$$

Algorithm 2, adapted from [36], shows the overall procedure on how to combine the ideas discussed until now. The number of episodes, α, γ, ε parameters are configured from the beginning. At each step in the episode, an action a is selected according to the ε-*greedy* policy, meaning that a has $\varepsilon \in [0,1]$ chances to be randomly chosen among the valid actions of state s. Moreover, with a probability of $1-\varepsilon$, a is chosen with respect to the policy found in the table, e.g. the action with the highest Q-value. It introduces some randomness in the agent's behavior that allows it to discover a more efficient sequence of actions, employing the exploration-exploitation trade-off already discussed. Another aspect worth mentioning is that Q-learning is an *off-policy*, which means that generating actions does not need to follow a specific policy, the agent will be able to learn even from a random behaviour.

Algorithm 2 Q-learning

```
1: procedure LEARNING(α ∈ (0,1), γ ∈ [0,1])
2:     Initialize the Q-table
3:     for each episode do
4:         s ← initial state
5:         while s not terminal state do
6:             a ←choose an action using ε-greedy policy from A(s)
7:             s', r ← apply a on the environment
8:             Q(s,a) ← Q(s,a)(1−α) + α[r + γ max_a' Q(s',a')]
9:             s ← s'
```

Q-learning is important from a theoretical point of view, because it was the first reinforcement algorithm that is guaranteed to converge to the optimal policy for MDPs [40], but under some conditions:

1. There exists an infinite number of observations (s_t, a, r, s_{t+1}) available, which implies that each pair (s_t, a) should be visited infinitely often. Formally, this translates in ensuring that any action a, from the available valid ones of a state s_t, has a non-zero probability of being chosen by the policy. In practice, it can be employed with the $\varepsilon - greedy$ policy

2. The learning rate α is strictly positive and must monotonically decrease and get close to 0, when the number of observations tends to infinity.

3. The sum of learning rates α_n, where $n \to \infty$ is the number of observations, should tend to infinity.

The last two points basically mean that the learning rate should converge to 0, but not very rapidly. This property can be formalized as: the sum of all α_n, $n \to \infty$, diverges, while the sum of α_n^2 should converge. Such an example is $\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \ldots \frac{1}{n}$.

We note that the above restrictions guarantee Q-learning will reach an approximation of the optimal policy π^*, that is, the actual policy π will be arbitrarily close to π^*, after an arbitrary number of steps. At the same time, the restrictions do not indicate anything about how fast π converges to π^*.

3.2.5 SARSA

SARSA, initially proposed in [29] as *Modified Connectionist Q-Learning*, is a RL algorithm that aims to approximate the optimal policy of an agent which interacts with an unknown environment, shortly, to solve an MDP. Like Q-learning, it exploits the *temporal difference* of samples and updates the Q-values of $(state, action)$ pairs, instead of just states, like TD(0) does. A distinctive feature of this approach is learning *on-policy*, meaning that the agent applies the estimated policy also to select actions. In other words, the policy that generates the behavior and the one updated are the same.

The name (SARSA) comes from the experience format: $\langle s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1} \rangle$, denoting that in the current state s_t, the chosen action a_t brought a reward r, transitioning to state s_{t+1}, in which action a_{t+1} is selected. Before performing an update, all the values of the tuple need to be known, hence the name.

The novelty of this technique arose as an answer to the question whether, when updating some Q value $Q(s_t, a_t)$, choosing $max_{a'} Q(s_{t+1}, a')$ as the best future return for s_t, is the best solution (as happens in Q-learning). Another option would be selecting the target value in the update rule as $Q(s_{t+1}, a_{t+1})$, which is actually given by the same $\varepsilon - greedy$ policy, that gives the behavior of the agent. The intuition behind this occurs from the fact that, when running Q-learning, in its early stage, it is unlikely that the Q-values are the good ones, and even later, it is possible that most of them are overvalued. Therefore, if modifying Eq.3.7 as just explained, we obtain Eq.3.8.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \tag{3.8}$$

The algorithm is rather simple (see Algorithm 3), similar to Q-learning, but with the two modifications explained before: choosing the action of the next state is done with $\varepsilon - greedy$ policy, before applying the update rule, and then Q-values are changed with Eq.3.8.

Although SARSA is not complicated, the proof of convergence to the optimal policy is not so straightforward, but still succeeded in [32], there are several conditions to ensure it:

Algorithm 3 SARSA algorithm

procedure LEARNING($\alpha \in (0,1)$, $\gamma \in [0,1]$)
    Initialize randomly the Q-table
    for each episode **do**
        $s \leftarrow$ initial state
        $a \leftarrow$ choose an action from s, using a policy derived from Q(ε-*greedy*)
        while s not terminal state **do**
            $s', r \leftarrow$ apply a on the environment
            $a' \leftarrow$ choose an action a' from s', using a policy derived from Q(ε -*greedy*)
            $Q(s,a) \leftarrow Q(s,a)(1-\alpha) + \alpha[r + \gamma Q(s',a')]$
            $s \leftarrow s'$
            $a \leftarrow a'$

1. Storing the Q-values is done in a look-up table.
2. All convergence conditions of Q-learning.
3. The initial $\varepsilon - greedy$ policy should become a greedy policy, in the limit, with a probability 1.

The last condition is due to the fact that SARSA is an on-policy approach, so the convergence of the policy π depends on the relationship between π and Q. In order to iteratively increase the dependence between them (π acting entirely based on the Q function, so as a greedy policy), ε needs to decrease gradually and approach 0, in contrast with Q-learning, where ε can be constant.

3.2.6 Q(λ)

In [28], an improvement for the convergence of classical Q-learning was proposed, by combining it with a technique typical of Monte Carlo methods and that is *eligibility traces*. In fact any temporal difference RL algorithm can be enriched with eligibility traces, leading to a new family of algorithms, called Q(λ). The intuition came from the fact that Q-learning propagates the reward and Q-value only to the previous state (it could be called Q(0)-learning), so it was considered that back-propagating them to the whole previous sequence of $(state, action)$ pairs, from the current episode, may reduce the time to converge. Therefore, the eligibility trace will stack these pairs, but only for actions selected by a greedy policy and this is required because Q(λ) is an off-policy method. Let's consider, for example, a series of greedy chosen actions $a_t \dots a_{t+k}$. If a_{t+k+1} is selected with a non-greedy policy, then no action from the trace should be updated, since the last one was generated by a different strategy. These being said, there are two cases when the eligibility trace is cleared: as just explained, when an exploratory action (e.g. random) comes next, or a terminal state is reached. This can be visualised in Figure3.6, found in [36].

Intuitively, disseminating the current Q-value, at each step of the agent, to all the past actions, could improve the convergence time especially for sparse reward use-cases. The propagation of a reward down the eligibility trace is done progressively, and controlled by $\lambda \in [0,1]$ factor. If $\lambda = 0$, then the algorithm becomes the classical Q-learning, otherwise if $\lambda = 1$, it

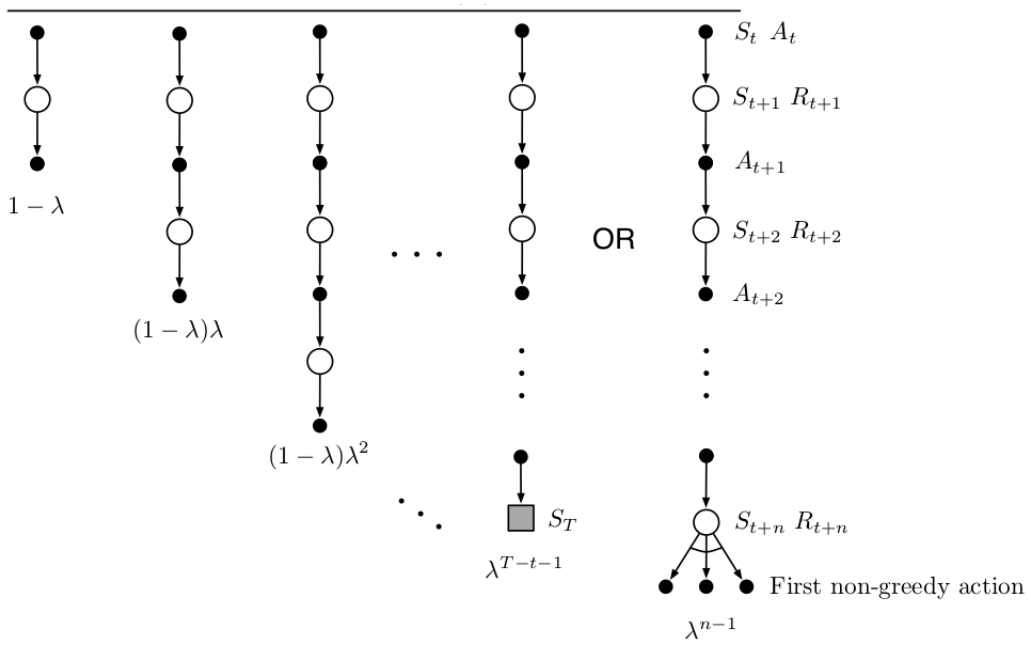

Figure 3.6: Eligibility traces for Watkins's Q(λ). The sequence of $(state, action)$ pairs ends either at the end of the episode, or when a non-greedy action is chosen.

will act as a Monte Carlo method. Q(λ) can be simply obtained by modifying Q-learning and adding the functionalities previously exposed (see Algorithm 4, adapted from [2]). Each episode starts with a zero-initialized eligibility table (lookup table for all $(state, action)$ pairs) and an empty list to store the eligibility trace. As seen until now, until the end of the episode, at each step, the agent chooses an action with an $\varepsilon - greedy$ policy. If the action is randomly chosen, the trace is cleared, and the eligibility of the current state and action becomes 0. Then, the new pair (s,a) is added to the trace and its associated eligibility is increased. For this we chose to increment it by 1, but other options to augment it can be adopted. The temporal difference value of the current state (δ) is computed as usual (line 17) and then applied to each action from the trace, discounted using the learning rate and the corresponding eligibility. At the same time, in the loop, the eligibility is reduced by λ and γ - the importance given to the next state's Q-value (line 18).

There are some drawbacks of Q(λ). Although it is presumed that it brings some improvement over Q-learning, and it was proved to converge empirically [38], there is no theoretical proof to formalize this. Besides it, the choice of λ parameter still needs to be fine-tuned experimentally. Another inconvenient is the use of eligibility traces, which requires updating the Q-values multiple times for each transition of the agent, so it may be too computationally expensive for large state spaces.

We discussed about Monte Carlo Methods in Section 3.2.3. Having Q(λ) algorithm implemented, by setting λ parameter to 1, the eligibility of each $(state, action)$ pair is not affected, so the *intensity* of changing $Q(s,a)$ values is not decayed across the sequence of state and actions. This change will lead to a Monte Carlo method. On the contrary, if $\lambda = 0$, the eligibility trace is applied only to the immediate $Q(s,a)$ value, hence Q(λ) becomes classical Q-learning.

Algorithm 4 Q(λ) algorithm

procedure LEARNING($\lambda \in [0,1]$, $\alpha \in (0,1)$, $\gamma \in [0,1]$)
  Initialize Q-table randomly for all $s \in S, a \in A(s)$
  for each episode **do**
    $E(s,a) \leftarrow$initialize eligibility table with 0s for all $s \in S, a \in A(s)$
    $sa \leftarrow$ empty list of state actions pairs
    $s \leftarrow$ initial state
    while s not terminal state **do**
      $a \leftarrow$ choose an action using ε-*greedy* policy
      if a is selected randomly **then**
        $E(s,a) = 0$
        $sa \leftarrow$ reset sa to empty list
      Append (s,a) to sa list
      $s', r \leftarrow$ apply a on the environment
      $E(s,a) \leftarrow E(s,a) + 1$
      $\delta \leftarrow r + \gamma max_{a'} Q(s',a') - Q(s,a)$
      for each (s,a) in sa **do**
        $Q(s,a) \leftarrow Q(s,a) + \alpha\delta E(s,a)$
        $E(s,a) \leftarrow \gamma\lambda E(s,a)$
      $s \leftarrow s'$

Eligibility traces for SARSA

As Q(λ) was derived from Q-learning by introducing eligibility traces, the same can be applied to SARSA. The resulting algorithm, SARSA(λ) will conserve the mechanism of updating Q-values, based on future actions, chosen using the same policy as the one for sampling the state-space (behavioral policy). For this, SARSA(λ) is also an on-policy method. It would be redundant to present the pseudo code, as it can be obtained easily by combining SARSA (Algorithm 3) with eligibility traces added on top of Q(λ) (Algorithm 4). More details can be found in [36].

3.2.7 Importance of the reward function and initial Q-values

A significant topic in RL is the choice of the reward function and this task remains in charge of the user. This subject is addressed particularly in [24], together with how to initialize the Q-values, to improve the overall algorithm performance. One straightforward method to reward the agent is by designing a binary reward function, as in Eq.3.9.

$$\forall s \in S, \forall a \in A(s), R(s,a,s') = \begin{cases} r_g & \text{if } s' = s_g \\ r_\infty & \text{else} \end{cases} \tag{3.9}$$

where s, s' are the current and the next state, a is the agent's action, r_g is the reward if the reached state is terminal and r_∞ is the reward for all the other transitions. Noting with Q_∞ the Q-value of a (s,a) pair in the limit, when $s \to s_g$, several situations of initializing the Q values (Q_i) can be analyzed, such as:

- $Q_i > Q_\infty$: a state already visited will have a lower Q-value, so in the next steps, non-visited states will come next (desired behavior).
- $Q_i < Q_\infty$ a state s already visited will have a higher value than those which were not, so in the next iterations, s will be visited again. In other words, at the beginning of training, the agent does not explore much, so learning will be slower. This behavior is called *moving round in circles*.
- $Q_i = Q_\infty$ states have the same Q-values before and after visiting, leading to a random behavior in the early training.

The experiments showed that, for $Q_i > Q_\infty$, learning was considerably faster than the other cases. Also, continuous reward functions inspired by the gaussian function can be imagined as proposed [24]. In the current work, the binary reward function is powerful enough for the agent to learn, but it can be enhanced, as we will see later in the report.

The conclusion is that, both the reward function and the initial Q-values have an impact on learning, but only at the beginning of training, therefore, their careful choice can improve the convergence time, but not the overall policy quality, if there are no time restrictions.

— 4 —
Experimental Methods

At this point, the reader has the prerequisite knowledge to introduce our approach on how to learn the execution of a domain specific language. As the task was envisioned by us, the solution starts with describing how to build from scratch a DSL in a clear and structured manner, so that it can be easily integrated with reinforcement learning algorithms and easily applicable for furthermore use-cases. The discussion will be organized around the case study called *Self Driving Cab*. In the first section, we present the meta-model, then its B specification and how we designed the reward function, continuing with Meeduse and ProB interaction. The second one will focus on showing some measurements regarding each algorithm after training, but mostly concentrated on key aspects that influence the quality and speed of learning, like designing a good reward function and choosing to precompute the state space or to discover it.

The whole B specification is split in three parts: static and dynamic semantics and the reward function. Throughout the chapter, the reader should have in mind Figure 4.1. The agent loads only the reward specification, which includes the dynamic semantics (these two are specified by the user), and the latter includes the static semantics, that was previously obtained using Meeduse, by injecting a model in the meta-model.

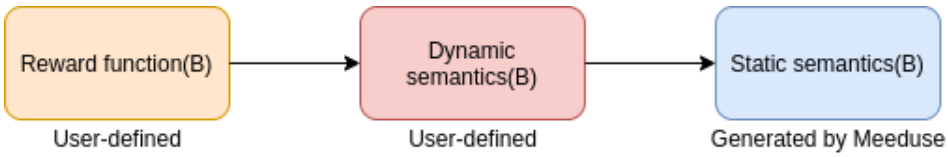

Figure 4.1: B specification - organization

4.1 Case Study - Self Driving Cab

We propose *Self Driving Cab* case, a reinforcement learning problem adapted by us from a previous version of it [9], which falls in the category of *grid world* tasks. The main feature of them is that the environment is imagined as a map with a grid-like structure. The agent, which can be placed on any position from a predefined set of possible coordinates, should find the path that maximizes the total reward to a predefined point on the map, by getting incentives or penalties.

The problem proposed by us is the following: there is a map with $m \times n$ positions (as in Figure4.2), which contains two objects: a cab and a client, each of them at different locations.

The client wants to reach a specific square on the map, called *destination*, and can only travel by taxi. The taxi can move on the map using four commands: NORTH, SOUTH, EAST, WEST, one square at a time, plus two additional indications: PICK-UP (to tell the client to get inside the cab) and DROP-OFF (to signal the arrival to the destination). Locations on the map can contain walls on any of the four edges, therefore, they could obstruct the taxi from advancing. The task is the following: the client needs to travel as quickly as possible, hence the taxi should cover a minimum number of squares in its ride, both to reach the client and the destination, in order to get a good feedback.

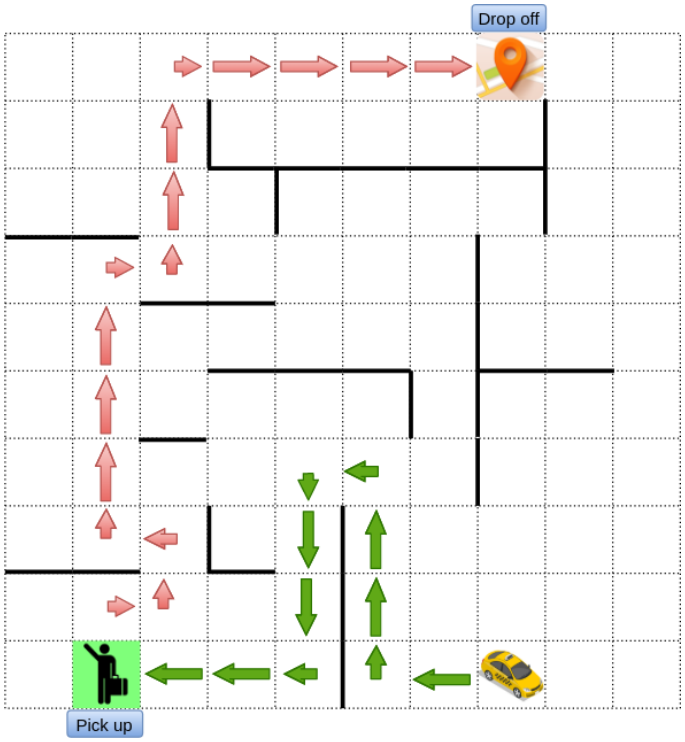

Figure 4.2: Self Driving map. The arrows show one of the the optimal paths to reach the client (green), then to transport him at the destination (red).

4.1.1 Meta-model

Furthermore, we depict how the model of the environment should look like, by describing one of its possible meta-models. Before, let's identify the concepts of the domain:

- *Map*: grid containing all the locations, the taxi and the passenger;
- *The set of locations*: a collection of equal size cells, which can be empty or contain a taxi and/or a passenger. A cell can have obstacles on each of the four sides;
- *Taxi*: the only mobile on the map, directly altered by the actions of the DSL;
- *Passenger*: can move only if picked up.

The design was done in Eclipse Modelling Framework, and can be seen in Figure 4.3. The root item is called *SelfDriving* and it can be imagined as the frame of the map, which encapsulates all the other elements. There can be only one taxi (class *Taxi*) and several passengers on the map, each of them in a specific location (class *Location*). A passenger (class *Paseenger)* contains an attribute *picked*, which is *false* before performing the PICK-UP command and *true* after. *Location* has information about the square coordinates on the map, plus several boolean attributes to indicate the presence of the walls and if it is a destination or not.

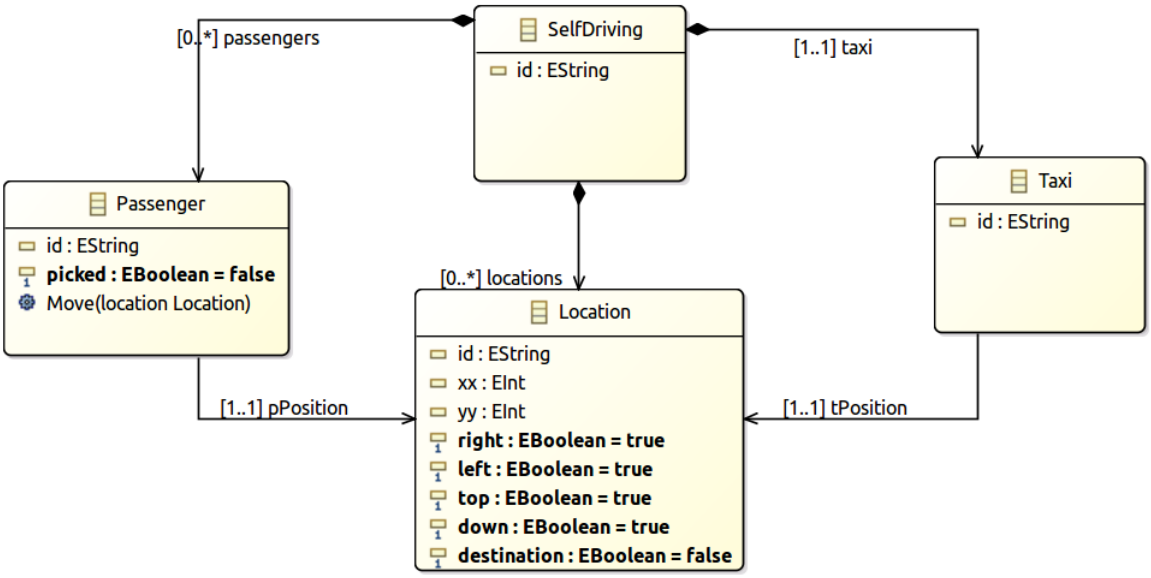

Figure 4.3: Self Driving meta-model

Up to this point we have explained the structure of the DSL, also known as the static semantics, contained in the meta-model. This is then processed in two ways: on one hand it serves for the creation of the model, on the other hand, to be translated in a B specification of the static semantics by Meeduse.

4.1.2 Static semantics

Using Meeduse, the meta-model designed in EMF is translated into an equivalent B specification. Figure 4.4 shows the sets and abstract variables obtained, while Figure 4.5 contains the invariant. The translation consists of converting EMF specific elements to B constructs. For example, Ecore classes and enumerations become abstract sets in B; basic data types, such as boolean, int, become B types (Bool, Z); attributes and references turn to functional relations. For more information on B concepts, check [1].

The abstract machine generated by Meeduse incorporates some *rudimentary* behavioral elements, such as: constructors, destructors, setters, getters. Furthermore, by employing the *Injector* component from Meeduse, the model is inserted in the B static specification. This means that the values contained in the model are inserted alongside their abstract definitions specified in B.

MACHINE

selfdriving

SETS

SELFDRIVING;
LOCATION;
PASSENGER;
TAXI

ABSTRACT_VARIABLES

SelfDriving, *Location*, *Passenger*, *Taxi*, *locations*, *pPosition*, *passengers*, *tPosition*, *taxi*, *Location_xx*, *Location_yy*, *Location_right*, *Location_left*, *Location_top*, *Location_down*, *Location_destination*, *Passenger_picked*

Figure 4.4: Sets and variables of the meta-model.

INVARIANT

$$
\begin{array}{l}
SelfDriving \in \mathcal{F}\ (SELFDRIVING) \land \\
Location \in \mathcal{F}\ (LOCATION) \land \\
Passenger \in \mathcal{F}\ (PASSENGER) \land \\
Taxi \in \mathcal{F}\ (TAXI) \land \\
locations \in Location \nrightarrow SelfDriving \land \\
pPosition \in Passenger \rightarrow Location \land \\
passengers \in Passenger \nrightarrow SelfDriving \land \\
tPosition \in Taxi \rightarrow Location \land \\
taxi \in SelfDriving \rightarrowtail Taxi \land \\
Location\_xx \in Location \nrightarrow \mathcal{Z} \land \\
Location\_yy \in Location \nrightarrow \mathcal{Z} \land \\
Location\_right \in Location \rightarrow \mathbf{BOOL} \land \\
Location\_left \in Location \rightarrow \mathbf{BOOL} \land \\
Location\_top \in Location \rightarrow \mathbf{BOOL} \land \\
Location\_down \in Location \rightarrow \mathbf{BOOL} \land \\
Location\_destination \in Location \rightarrow \mathbf{BOOL} \land \\
Passenger\_picked \in Passenger \rightarrow \mathbf{BOOL}
\end{array}
$$

Figure 4.5: Invariant of the meta-model.

4.1.3 Dynamic semantics

We enumerated above the six available operations of the taxi driver, and now we see what is the actual behaviour behind them. Each command is a rule implemented in B language and satisfies two main requirements. On the one hand, it expresses what the execution does, and on the other, what are the safety requirements to be fulfilled by the agent. Before continuing to the implementation, let's identify these requirements. First, as the map contains cells with obstacles, the taxi must avoid them. Second, the driver is allowed to pick-up the passenger only if both of them are in the same position, and lastly, the driver cannot drop-off the passenger

anywhere on the map, but only at a location marked as the destination.

Figure 4.6 contains the B specification for *south* and *drop-off* operations. Predicates on line 4 and 5 give the next position of moving to south, by incrementing the X coordinate, while the predicate on line 6 makes sure the taxi does not go into a wall. Drop-off rule ensures that the target location is a destination, the taxi is located on it and the passenger is picked-up. This kind of rule is useful, because it considerably restricts the space of unsafe drop-offs in locations that are not destinations. For example, adding the condition for the passenger to be picked-up, before dropping-off, reduces the number of possible states by $10 \times 10 = 100$. Additionally, restraining the drop-off only on a destination location reduces the state space by $10 \times 10 - 1 = 99$. The specification for all operations can be seen in Figure A.1.

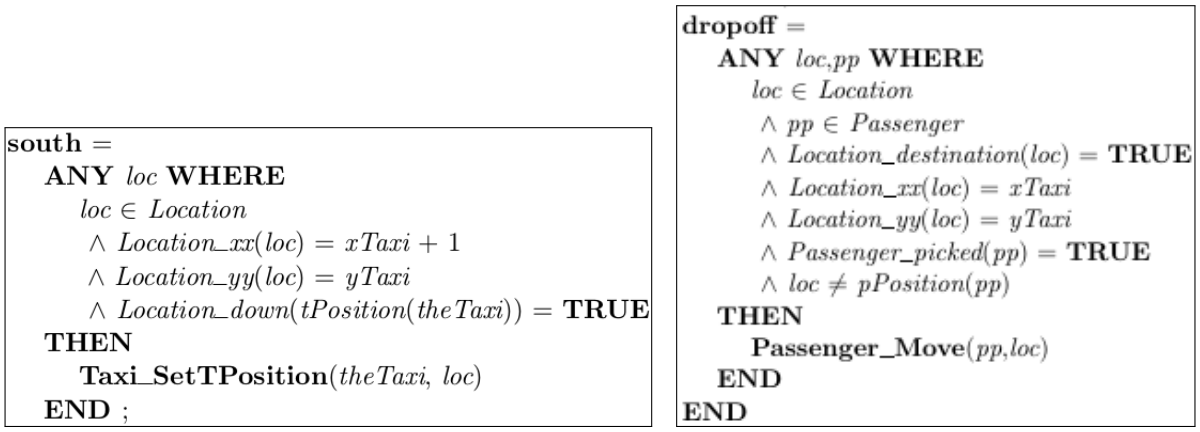

Figure 4.6: Formal specification of the rules *south* and *drop-off* in B.

4.1.4 Reward function

One other important aspect to be discussed is the reward function, which can also be written in B, on top of the dynamic semantics. The reward is designed depending on the goal that the developer wants to be achieved by the agent. For the current use-case, the purpose of the taxi driver is mainly to drop-off the passenger at the destination location, so it is considered a positive action and rewarded with a high reward (e.g. 20). Similarly, even though the pick-up is not the final goal, we can consider it a partial objective, hence it also benefits from a high reward. Any other action of the remaining four (*north*, etc.) is considered to move the taxi away from the target location, so we assign it a negative reward (e.g. -1). The most straightforward reward function in our case is a binary function, with feedback associated to each action type, Eq.4.1.

$$R_1(a) = \begin{cases} 20 & \text{if } a = \text{drop-off}, \\ -1 & \text{else} \end{cases} \tag{4.1}$$

Figure 4.7 contains a sample of how rewards can be associated for each action.

An improvement over a reward function is to customize the feedback for the actions according to the expected quality of the action. For example, in R_1, the *pick-up* can be rewarded also with 20 (Eq.4.2).

```
reward ← south =
  BEGIN
    move.south ||
    reward := -1
  END ;
```

```
reward ← dropoff =
  BEGIN
    move.dropoff ||
    reward := 20
  END
END
```

Figure 4.7: Example of reward definition in B for *south* and *drop-off*.

$$R_2(a) = \begin{cases} 20 & \text{if } a \in \{\text{pick-up, drop-off}\} \\ -1 & \text{else} \end{cases} \tag{4.2}$$

The reason for rewarding with a small quantity the four operations (*north*, etc.), like -1, is to penalize an excessive number of moving steps to reach the terminal state. In this way, the agent will be forced to find a sequence of steps that minimizes the path to reach the destination, implicitly that maximizes the cumulative reward. We do not claim that these values are the most suited, but intuitively, they have potential to give a solution.

A great advantage of designing the reward function in B (and the dynamic semantics in general) is the ability to specify the properties to be satisfied, rather than to encode information. For example, in a general purpose language this is rendered by nested *if* ... *then* ... *else* ... instructions, which can be tedious to develop and maintain for complex functions. This can be shown for a rewarding strategy intuitively better than R_1 and R_2, as follows: when the taxi is located in a location close to the pick-up/drop-off point, it can receive a reward proportional with the distance between them. It means that the closer the taxi is to the destination, the higher the reward. For a distance of 1, the rewarded locations can be seen in Figure 4.8. In Figure A.2 we see the B specification of a function that rewards the agent whenever the taxi is close to the destination.

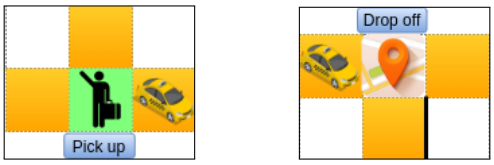

Figure 4.8: Rewarding based on proximity. When the taxi is close to the passenger (e.g. yellow locations), it is positively rewarded.

During the learning process, when running one of the algorithms presented in the previous chapter, the agent emulates the interaction with the environment by calling ProB API, which provides at each step the set of available actions, animates the transitions between states and returns the reward. As already mentioned, ProB can be replaced by a DOT file that contains the whole graph state space computed in advance.

4.2 Results and Discussion

In the coming section we present several results related to the previously described algorithms (SARSA, Q-learning, Q(λ), SARSA(λ), Monte Carlo), then we try to formulate some generic

observations that may be useful for other use-cases. First, several plots will show the overall performance of each algorithm. Note that all of them depend on hyperparameters (α, γ, ε, λ) which normally need to be fine-tuned for the problem at hand, but this would be out of our main focus, some "standard" values will be used, in other words, that are likely to be close to the optimal ones. Next, we will see the performance of two variants of rewarding the agent and which algorithms could avoid being affected by a poorly designed reward function. Later we see the results for several ways of initializing the values of the Q-table and finally a comparison of running time between using ProB to explore the state space or a pre-computed state space stored in DOT format. Interaction with ProB may be slower, and if this is confirmed, we will see how much.

4.2.1 Performance measurement

Studying how fast each algorithm learns can be done by plotting the total reward obtained by the agent until reaching the end of an episode, but most of all, this shows if the agent learns or not. All of them converged to the optimal sequence of actions in less than 1000 episodes (Figure 4.9).

The hyperparameters used are: learning rate $\alpha = 0.05$, discount factor $\gamma = 0.6$, $\varepsilon - greedy$ factor $\varepsilon = 0.6$, propagation decay parameter $\lambda = 0.7$. For Q-learning, the learning rate was not decreased as required in the theoretical convergence conditions, because the algorithm reached the optimal solution without this intervention. The reward function used is: *north, south, east, west* = -1, *pick-up* = 20, *drop-off* = 20, as explained in the Section 4.1.3. The cumulative reward signal is rather noisy, so a Butterworth filter was applied on it to eliminate high frequencies and obtain a flatter curve.

The original curves of both SARSA algorithms are more smooth than the other three, especially for later episodes, due to decreasing the ε value, which ensures the $\varepsilon - greedy$ policy to become a *greedy* policy. For Q-learning and Q(λ), the improvement using trace eligibility is not so obvious, but for SARSA and SARSA(λ) it is. It can be easily noticed that for SARSA(λ), the curve becomes smooth earlier than for SARSA.

Monte Carlo also converges rapidly, similarly to the other methods. In Figure 4.10, the filtered cumulative return curves can be seen for all five algorithms. Probably, the fact that our model is not very large ($\approx$200 states) does not allow an obvious distinction, but still, at the lowest point, Q-learning can be distinguished with the lowest total reward evolution curve, and on the other hand, SARSA(λ) seems to have the best cumulative reward graph.

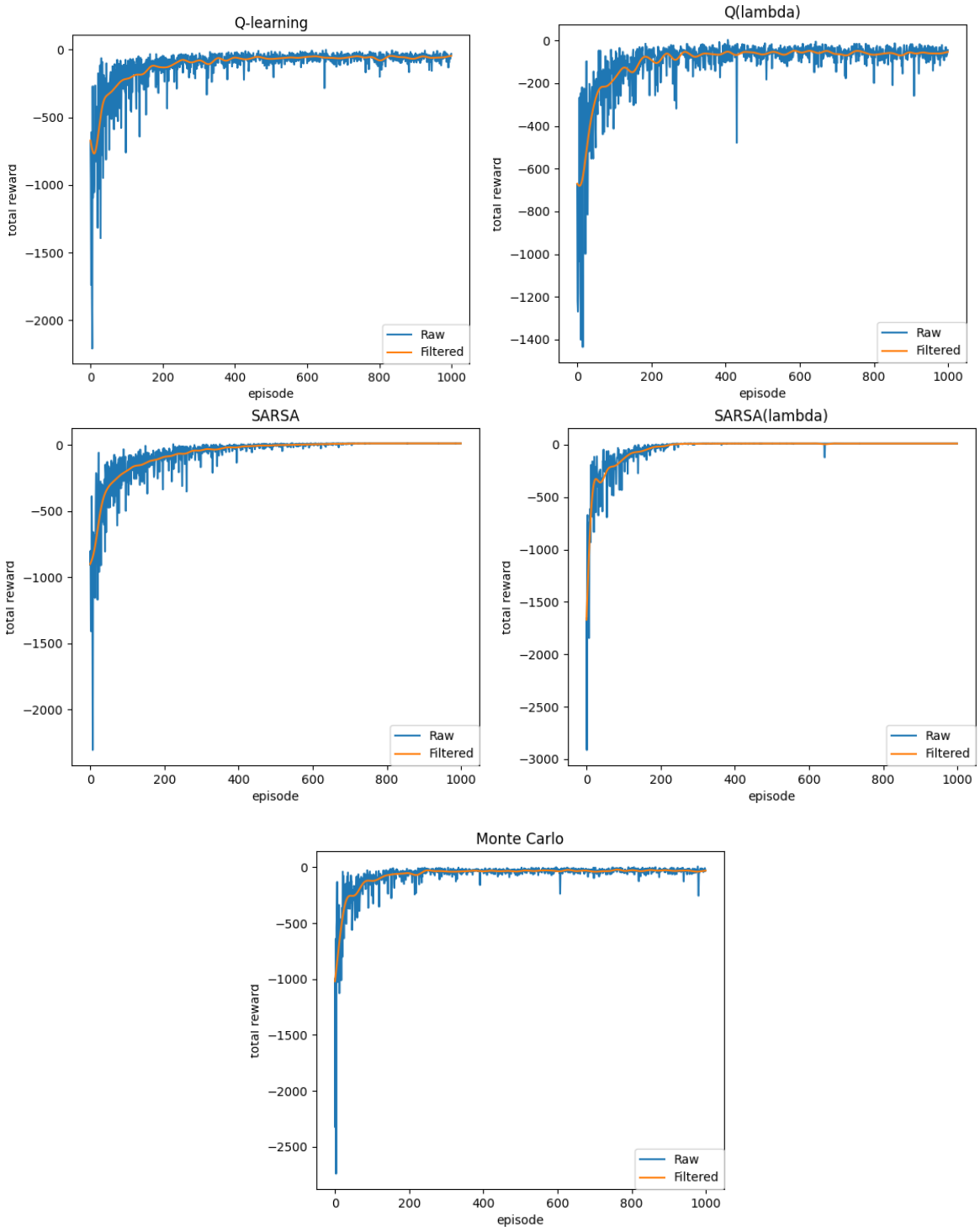

Figure 4.9: Q-learning, Q(λ), SARSA, SARSA(λ) and Monte Carlo cumulative reward over episode.

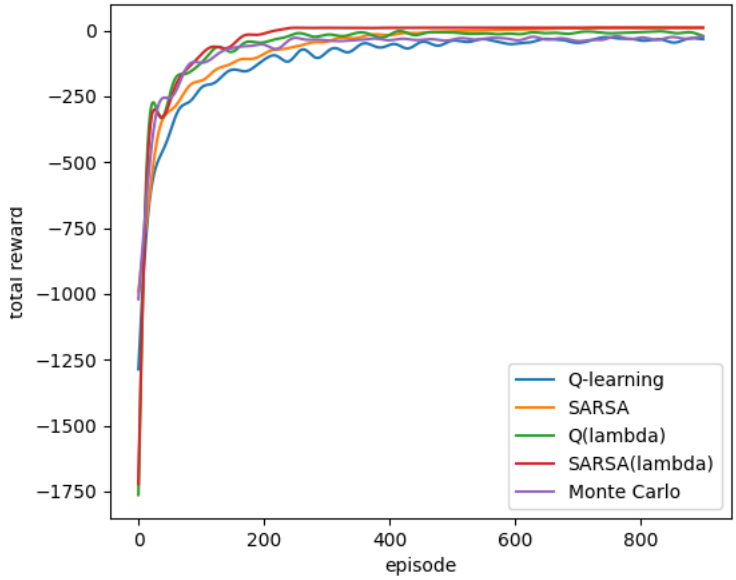

Figure 4.10: Cumulative reward graphs of all five algorithms.

4.2.2 Comparison of reward functions

Next, we see how the choice of the reward function can impact the speed of learning. For the experiment, we will refer to the functions proposed in Subsection 4.1.3: R_1, R_2, R_3. For R_1, SelfDriving use case could be seen as an example of sparse rewarding system, because, most of the time, the agent is penalized with -1 for any of the four directions (*north*, etc.) and only one time per episode with +20 for *drop-off*. We consider the second reward function, R_2, derived from R_1 by modifying the feedback for *pick-up* operation from -1 to +20, hence the reward becomes *less sparser*. Intuitively, learning should be considerably hampered for R_1, compared to R_2. Regarding R_3, we try to add an additional positive feedback (specification in Figure A.2), when the taxi is close to a target location. Training is performed with the same hyper-parameters as above, over 1000 episodes, and for each algorithm the total reward/episode is plotted for all R_1, R_2 and R_3.

In Figure 4.11, the evolution of the cumulative return curve after filtering is plotted for all five algorithms. The first aspect worth noticing is that the learning curves for R_2 are below the curves for R_1 every time, and their advancement is more moderate than for R_1. That is to say, the agent's policy converges significantly slower to the optimal one, so our initial assumption is confirmed. For all algorithms, R_3 does not show an obvious improvement, compared to R_2, both of them having almost the same evolution. We were expecting to see a faster convergence for R_3, but our supposition is that the size of state space is not so large to highlight this aspect.

In Section 3.2.6 we explained how employing trace eligibility mechanism is inclined to improve learning especially for sparse rewarding models. For Q-learning and Q-lambda it is not so obvious (maybe if taking a closer look at Figure 4.10), but regarding SARSA and SARSA(λ) algorithms, it is visible that the latter it is not so affected when having a sparse reward function. At the beginning of the training, the total reward curve for R_2 starts at a very low point, but it recovers rapidly and after $\approx$300 episodes catches the one for R_1. On the other hand, SARSA takes $\approx$1000 episodes for R_2 to get close to R_1.

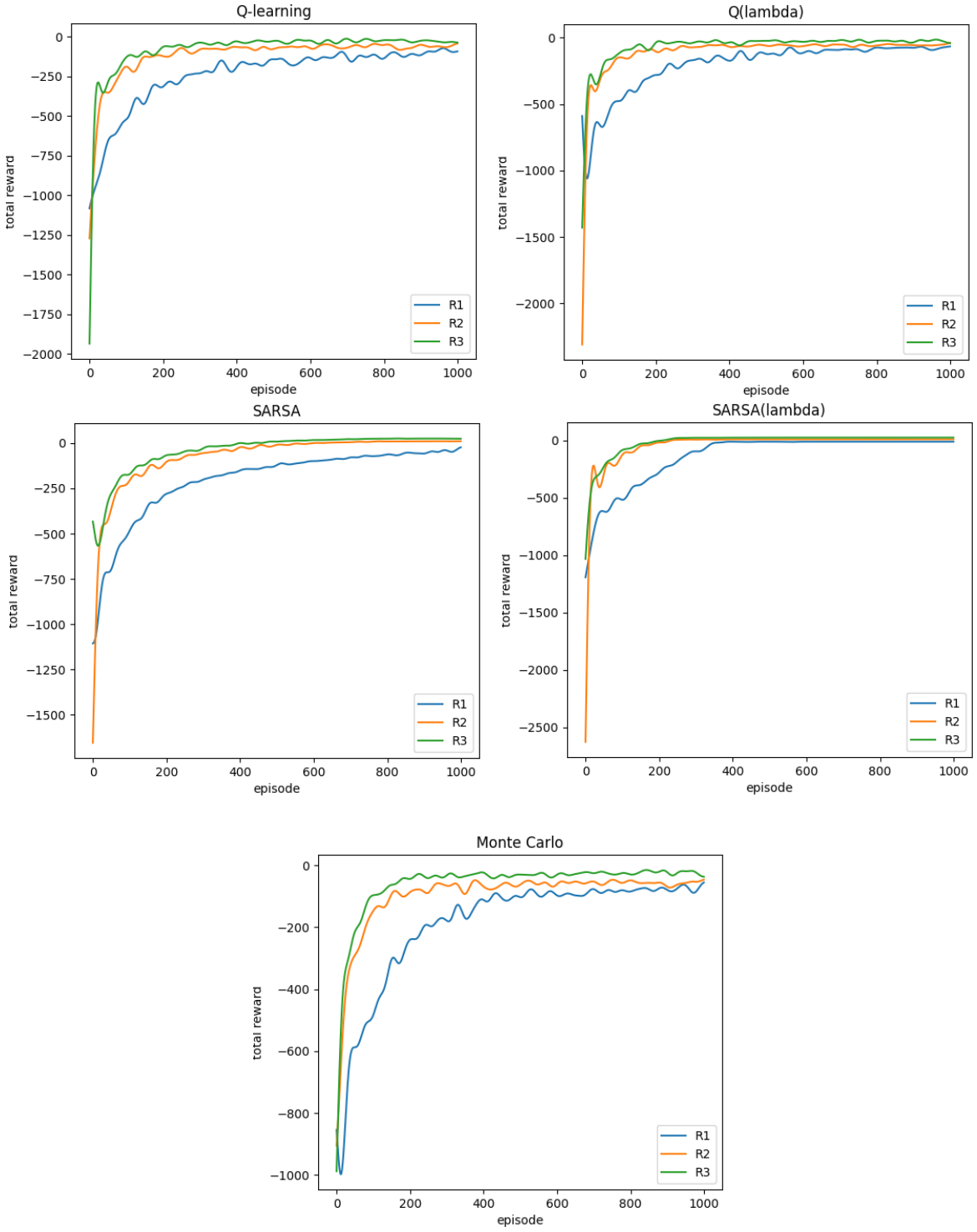

Figure 4.11: Q-learning, Q(λ), SARSA, SARSA(λ) and Monte Carlo trained for three reward functions.

4.2.3 Initial Q-values impact on convergence

Furthermore, we will see how several types of initializing the Q-table can influence the convergence, but only at the beginning of the training. As discussed in Section 3.2.7, initial Q-values should be chosen depending on (Q_∞), which according to [24], it can be computed as:

$$Q_\infty = \frac{r_\infty}{1-\gamma} \tag{4.3}$$

For $r_\infty = -1$ and $\gamma = 0.6$, then $Q_\infty = -2.5$. Please note that this is valid only for $(state, action)$ pairs that lead to a non-terminal state. For terminal states, Q_∞ is different, but since their number is so small, we omit them. We tried some values $Q_i \leq -2.5$, but at the beginning of the learning process, the total reward plot is so small that it cannot be displayed across other more reasonable values, due to the very large dimension of the OY axis. Normally, we could have chosen any values greater than -2.5, so we selected -2.3, 0 and 20. The evolution of the cumulative return/episode can be seen in Figure 4.12 for Q-learning and SARSA, as representatives for off/on policy learning.

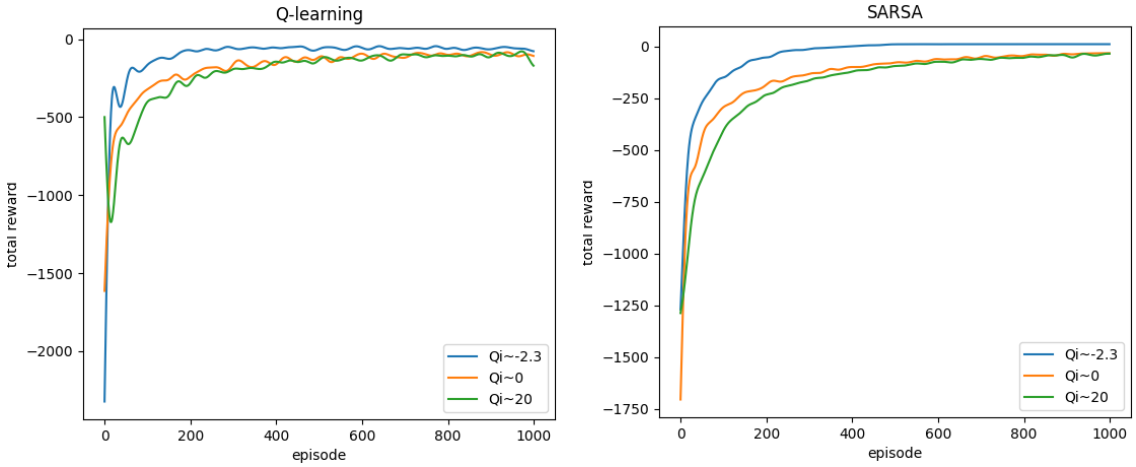

Figure 4.12: Q-learning and SARSA. Different ways of initializing the Q values.

We notice that the closer the initial Q-values are to $Q_\infty = -2.5$, the faster both algorithms reach the optimal policy. Even if the initial Q-values do not pose a threat to convergence, they should not be neglected, especially for DSLs with large number of states, since training time can be considerably reduced.

4.2.4 Discovered versus precomputed state space

For all the experiments performed until now, the state space transitions and rewards were obtained by communicating with ProB model checker, thus discovered while training. This scenario fits very well the description of the RL process, according to which, the agent interacts with the surroundings in order to learn, and for each action chosen by the agent, the environment transitions to a new state. At the end of an episode, the whole environment is restored to an initial state and the process will repeat until convergence. If taking into account that ProB spends computation time to discover paths of states which may have already been used by the

agent in a precedent episode, we can think that it is preferable to have the whole state space computed beforehand, so as to avoid repetitive transitions. Also, the implementation of ProB is done in Prolog and the communication with it takes place via sockets, hence, one may intuit that all this interaction is very likely to produce a significant undesirable overhead. To address these issues, we propose instead to use the state space computed in advance and stored in a DOT file. DOT represents a graph description language, which in our case can contain information about transitions between states, together with their associated reward. It should not be forgotten that obtaining a precomputed state space in DOT format requires the model to be checked, and in our case it takes ≈2.5s, plus another ≈2.6s to write it on disk. In Table 4.1 we show the time required by each algorithm to reach the optimal solution, for both two earlier described situations. The first column contains the approximate number of episodes needed by each algorithm to reach the optimal solution. It is interesting to notice that a high number of episodes until convergence does not imply a long period of time to train, e.g. SARSA. Nevertheless, we do not intend to do a comparison of algorithms, as it is not the purpose of the work, their performance depends very much on the use case, but to raise awareness that a small number of episodes is not correlated with the amount of training time. Even so, it depends on how the state space is unfolded for the agent. For example, as it can be seen in the table, for Q-learning and SARSA(λ), it takes more time to train when discovering the state space than to precompute it, while for Q(λ) and Monte Carlo it happens the opposite.

Algorithm	Nb. episodes	Discovered states (s)	Precomputed states (s)
Q-learning	450	7.14	0.026
Q(λ)	311	4.56	0.039
SARSA	600	5.95	0.018
SARSA(λ)	750	7.71	0.022
Monte Carlo	225	4.7	0.041

Table 4.1: The number of episodes and elapsed time until reaching the optimal solution obtained via discovering or precomputing the state space. Precomputing took additional ≈5.1s. The times for the last 2 columns are obtained averaging 10 different trainings.

Another aspect that should not be ignored is the size of the DSL. While for DSLs with a small state space, which can be extensively explored, it may be preferable to compute them only once and store them in DOT format, to be used whenever needed, for larger DSLs, the complete examination of the state space may be impractical, so a partial online exploration can be employed. It largely depends mostly on the user's needs.

The results and insights presented should not be seen as concentrated on reinforcement learning, but mostly to advertise the importance of some aspects when designing an executable DSL and learning its execution, like: choosing the learning algorithm, why to pay attention to the reward function and initial Q-values, using a precomputed state space or a discovered one. Overall, the results seen in this chapter are promising. The algorithms proved to be suitable to learn the execution of the proposed DSL. The fact that they do not require data acquired beforehand is a great plus, so it encourages us to continue using them for other use-cases. However, we bear in mind that Self Driving Cab is not a real world problem and its state

space is not so large, so eventual limitations of our approach may exist and be unknown at this moment. To conclude, our work is a proof that there is a high potential in integrating AI techniques in the MDE field, especially for tasks that can be automated, leading to the reduction of effort and time invested by users.

— 5 —

A Deep Learning approach to express the Q-table

This chapter can be seen as a natural continuation of the previously presented methods and results, but in fact it was the initial focus of our work, with applicability on a more complicated DSL, and this is chess. Why chess? Because it is a real problem which can help us to offer a proof of concept for the smart execution DSL task. At first, learning the execution of a DSL was thought of as predicting the movement of pieces on a map to achieve some goal. We were inspired by [27], which formulated the task of predicting moves in chess as a supervised machine learning problem split in two: training a neural network (NN) that learns to predict the piece to be moved, and a second one to predict the position where it should be placed. For such formulation, the issue is that any of the two NNs can choose invalid positions, that violates the rules of the game, so a contribution of us would have been on filtering the valid moves out of all the possible ones using MDE and B-method, but we reconsidered this option due to some inconveniences. One challenge is the representation of the board to be fed to both NNs, such that they can extract useful information from it. The chosen encoding in [27] was an 8x8x6 matrix of 0,1,-1, in which each piece is represented as a 6 bit array, and the color as negative/positive. The advantage of such a strategy is that it can be applied further to almost any DSL based on pieces. Just to keep the encoding general we chose to Others formulated chess as an image processing task, thus working at a pixel level [21]. Another work [26] defined several ATARI games as reinforcement learning problems, also with data represented as collections of pixels. Since our task formulation is based on a DSL and for each piece we retain only the location of pieces, the binary encoding is more suitable. At the same time, from a computational point of view, this representation is cheaper to process, as the size of the board is considerably smaller than that of an image. Nevertheless, when dealing with supervised machine learning tasks, a problem is the need of a dataset composed of games played in advance. For these reasons, we directed our attention to more general methods, which do not require state encoding or training data, as the ones presented in the previous chapters, in particular to Q-learning.

Compared to Self Driving Cab, chess has a distinction that makes it particularly challenging, and this is the huge number of possible states ($\approx 10^{46}$). Consequently it becomes impractical to store the Q-values in a tabular data structure. To solve the limitation, we come back to the definition of a Q-table, which is a mapping from $(state, action)$ pairs to Q-values. Because this basically determines a function, it can be replaced with a *neural network*, also known in the literature as *universal function approximator*. There is much to say about NNs [15], but we limit

ourselves to say that a neural network is a structure of nodes organized in layers and weighted edges, that is capable of learning mappings between multidimensional numerical inputs and outputs, by feeding them a sufficient number of samples.

One aspect to be clarified is how Q-values are extracted from a NN. For a Q-table, it is simply done by indexing the entries using the states and actions IDs, but a NN does not allow this. Another problem that arises for chess and possibly for other large DSLs, is how to define an action. Self Driving Cab has only 6 actions, but chess many more and it is not straightforward to describe them. These issues can be solved using the intuition that a $(state, action)$ pair can be interpreted as the next state of the board that results by applying the action on the previous state. Hence, we can think of a NN as a function which computes the Q-values for states and the agent can choose the best action that leads to a state with the highest Q-value.

Two more components need to be discussed: the DSL and the reward function. Expressing both of them in B requires to invest a non-negligible amount of time so we oriented towards existing solutions that could replace them, without making any compromises on the expected functionality. Regarding the DSL, we found a python library[1] that satisfies our needs. One exploited feature is, for example, the ability to perform actions on the board stored in memory (e.g. push valid moves) which will automatically transit to a next state. Another important aspect is the ability to provide at any moment in the game a list with all the valid available actions to the agent, from which will choose one with an ε-greedy policy. For chess, the reward function is intended to give a high positive feedback for reaching an advantageous state to the player, depending on the quality of its pieces on the board. In the literature it can be found with the name of *heuristic function*. One popular version is given by Stockfish engine[2], which we also chose to work with. Broadly speaking, Stockfish unfolds a valid-move tree and must score the resulting positions. The evaluation obtained can measure the number of moves until mate or the board value in *centipawns*. The centipawn measures the advantage of a position, where 1 pawn = 100 centipawns. The interaction between the agent which tunes the neural network, based on the valid moves provided by the DSL and the reward given by Stockfish can be visualized in Figure 5.1.

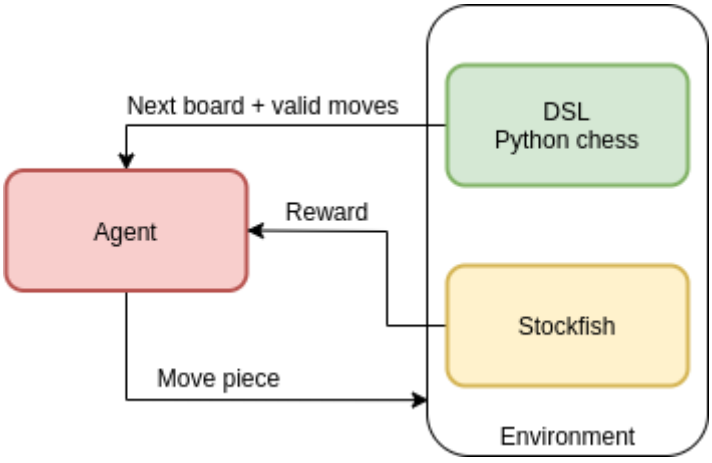

Figure 5.1: Chess DSL - Stockfish rewarding system - Agent interaction.

[1] https://python-chess.readthedocs.io/en/latest/
[2] https://stockfishchess.org/

The NN to approximate the Q-function is a *convolutional neural network* (CNN) whose important distinctive feature is the use of filters that employ the convolution mathematical operation. We used a network with two convolutional layers with 32 respectively 64 filters, followed by two fully connected layers. The size of the filters that we tried is 3×3 and 5×5. Each layer is followed by a non-linear activation, it is about the *Rectified Linear Unit*. There would be more to say on this, but the reader can refer to [30] for details, from which we inspired our architecture.

The algorithm broadly conserves the principles of Q-learning, with the mention that the Q-table is replaced by a NN, hence it becomes *Deep Q-learning*. A detailed explanation of it can be found in [26].

To experiment with the proposed approach, we ran the training for $\approx$ 40k episodes. For unknown reasons, we cannot provide conclusive results for the method, and the time constraints prevented us from studying more in-depth the task, in order to stipulate a sound opinion. However, we suspect some aspects which need more attention in a future work. One of them is to try other representations of the pieces on the board. Binary encoding is a straightforward one, so encodings that capture more relevant information about the elements of the board and the relation between them could be tried. Probably the most susceptible to failure is the neural network architecture. It usually takes lots of trial and error to tune its hyper-parameters with the purpose of reaching convergence, hence more effort should be invested also in finding a proper NN architecture. A similar approach to ours [31] uses NNs to approximate the Stockfish evaluation function for chess board positions obtained from games played in advance, hence the problem is easier, since it is formulated as a supervised machine learning task, and for this, we are optimistic about the overall approach of the strategy that it can be successfully applied on chess and other DSLs similar to it.

— 6 —
Conclusion and Future Work

The problem of predicting execution semantics of programs is still unsolved and currently under research. Our contribution in this area is mainly concentrated on how artificial intelligence methods can be applied throughout the design stages of a program, with a main focus on executable domain specific languages. The nature of the effort was predominantly exploratory, and this allowed us a great degree of freedom in choosing the most suitable AI techniques. At the beginning, we focused our research on neural networks, today a superstar of the AI domain, but after a better investigation of the problem, we realized that a more convenient representation of the task could be done using Markov Decision Processes. This mathematical framework came together with strong formalisms and extensively researched solutions as Q-learning, SARSA, Q(λ), SARSA(λ) and Monte Carlo algorithms, which are exponents of reinforcement learning philosophy.

Our work would have not been possible without Meeduse and ProB, which allowed us to design the DSL and its mechanisms of execution towards simulating the behavior or the program. The major mission of the tool was translating the DSL's meta-model into an equivalent B specification, in which an instance of the meta-model was injected. ProB was important, on the one hand, to formally check the abstract machine specification generated by Meeduse, using B-method, and on the other hand, to animate the transitions between states.

An inconvenience when defining a system is that the user must choose the operations to be executed. The main motivation of the thesis was to automate this process, in other words, to find a way to learn and then predicting the necessary actions to reach predefined goals. We envisioned this process as an agent which performs actions and learns a strategy by observing state transitions and their associated reward. The goal is defined in terms of a reward function designed by the developer in B language. It should not be forgotten that such a function is not hard-coded as in most of the cases, but specified based on domain's properties. Basically, *it tells the agent what skill to learn*.

The chosen case study, Self Driving Cab, allowed us to show how the theoretical approach envisioned can be practically achieved. We first checked using cumulative rewards plots per episode that the agent learns using the presented algorithms. After expressing the static and dynamic semantics, we explored how the goal can be achieved using different reward functions written in B. We then saw why it is necessary to invest effort in designing a good reward function, if the training time is a constraint. A discussion was done on the importance of the initial Q-values for improving the training time required by the algorithms to converge. Finally, we proposed an alternative for the interactive animation of ProB, which can be replaced by a precomputed state-space in DOT format.

We hope this successful attempt of integrating reinforcement learning techniques with MDE and formal methods will inspire other people to exploit the capabilities of AI in their work. One particular challenge is that the experts need to come together and discuss the needs for their projects, because it may not be obvious how one's domain could take advantage from solutions found in other fields.

A major limitation of the tested algorithms occurs for DSLs with a large state-space, because the learned information is stored in a Q-table. Since each row in the Q-table corresponds to a state and the columns represents the possible actions, hence they are dependent on the available computer memory. In the literature, this inconvenience can be addressed using neural networks, so we plan to continue working on trying to find general representations of the DSL's states, from which NNs can learn. We presented the context of such a necessity for chess, in Chapter 5.

Another direction of our efforts is heading at the same time towards a real-life problem, and this is Railway Systems. Ensuring the errorless execution of railway mechanisms is still a problem under development. However, the great advancements in the field of formal methods make it possible to formulate their requirements in mathematical notations, which can be then formally proved. In our team, it is currently under development such a DSL, for which, as in this report, the same methodologies are used to build it and prove its correctness. Therefore, we envision applying the techniques described in this thesis, in order to assure their safe execution. The success of such an application would confirm that our methods can be applied to other real-world problems.

— A —
Appendix

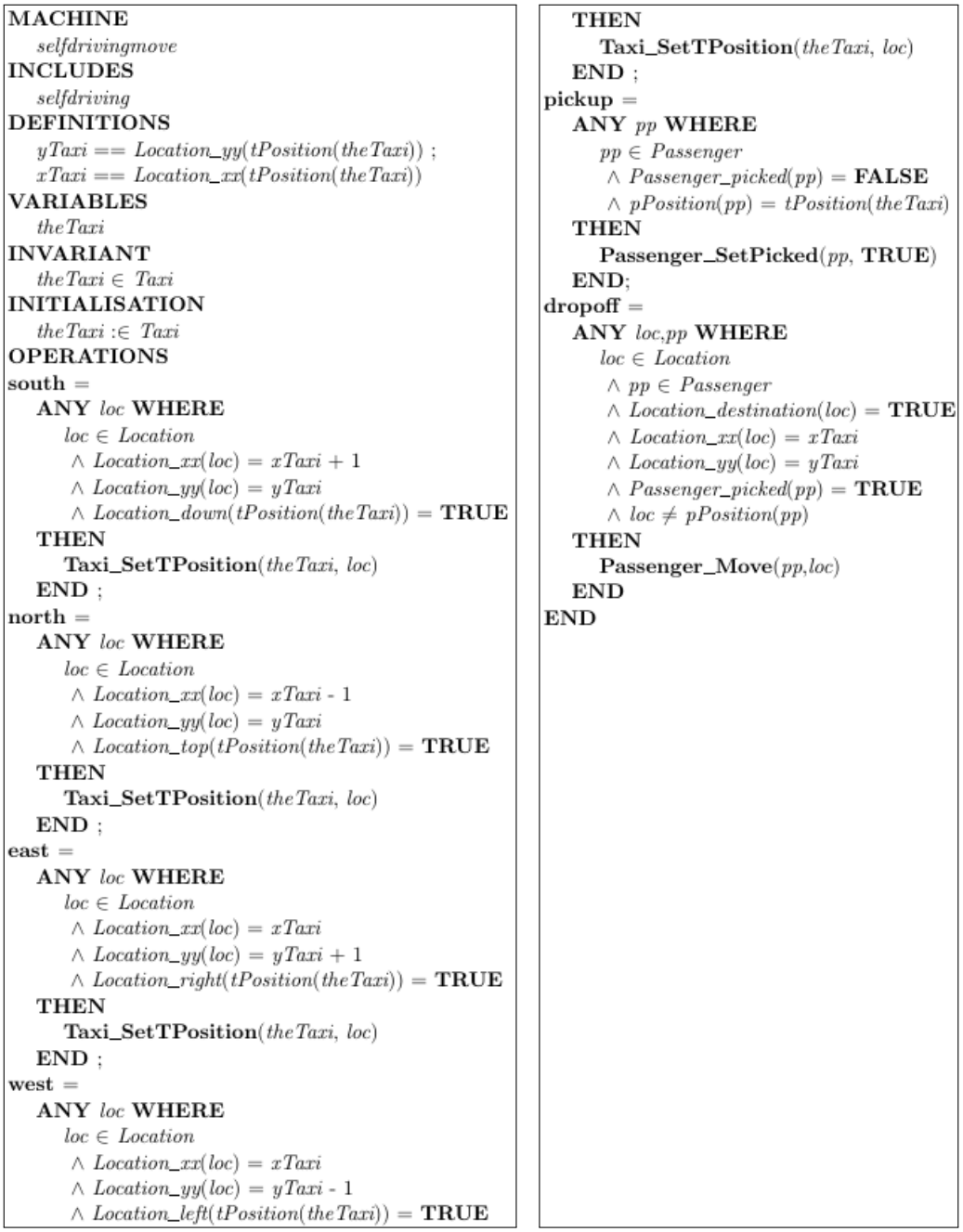

Figure A.1: Self Driving Cab - full dynamic semantics in B.

MACHINE
selfdrivingmainBis
INCLUDES
move.selfdrivingmove
DEFINITIONS
$xTaxi == move.Location\_xx(move.tPosition(Choupette))$;
$yTaxi == move.Location\_yy(move.tPosition(Choupette))$;
$xPassenger == move.Location\_xx(move.pPosition(Bob))$;
$yPassenger == move.Location\_yy(move.pPosition(Bob))$;
$near(distance) ==$
$xTaxi \in xPassenger - distance \,..\, xPassenger + distance$
$\wedge\ yTaxi \in yPassenger - distance \,..\, yPassenger + distance$;
$dest(distance) ==$
$\exists\ loc.(loc \in move.Location \wedge move.Location\_destination(loc) = \mathbf{TRUE}$
$\wedge\ move.pPosition(Bob) \neq loc$
$\wedge\ move.Location\_xx(loc) \in xTaxi - distance \,..\, xTaxi + distance$
$\wedge\ move.Location\_yy(loc) \in yTaxi - distance \,..\, yTaxi + distance$
) ;
$picked == move.Passenger\_picked(Bob) = \mathbf{TRUE}$;

$rewardFunc ==$
BEGIN
$reward := -1$;
IF $near(0) \wedge \neg\ (picked)$ **THEN**
$reward := reward + 5$
END ;
IF $near(1) \wedge \neg\ (picked)$ **THEN**
$reward := reward + 1$
END ;
IF $picked \wedge dest(0)$ **THEN**
$reward := reward + 5$
END ;
IF $picked \wedge dest(1)$ **THEN**
$reward := reward + 1$
END
END
VARIABLES
reward
INVARIANT
$reward \in \mathcal{Z}$
INITIALISATION
$reward := 0$
OPERATIONS
$rr \leftarrow$ **south** =
BEGIN
move.south ;
rewardFunc ;

```
        rr := reward
    END ;
rr ← north =
    BEGIN
        move.north ;
         rewardFunc ;
        rr := reward
    END ;
rr ← east =
    BEGIN
            move.east ;
        rewardFunc ;
        rr := reward
    END ;
rr ← west =
    BEGIN
            move.west ;
        rewardFunc ;
        rr := reward
    END ;
rr ← pickup =
    BEGIN
        move.pickup ||
        rr := 10
    END ;
rr ← dropoff =
    BEGIN
        move.dropoff ||
        rr := 20
    END
END
```

Figure A.2: Self Driving Cab - proximity reward function.

Bibliography

[1] Jean-Raymond Abrial and Jean-Raymond Abrial. *The B-book: assigning programs to meanings*. Cambridge university press, 2005.

[2] Angela Barriga et al. "A comparative study of reinforcement learning techniques to repair models". In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. 2020, pp. 1–9.

[3] Loli Burgueño, Jordi Cabot, and Sébastien Gérard. "An LSTM-based neural network architecture for model transformations". In: *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE. 2019, pp. 294–299.

[4] Albert F. Case. "Computer-Aided Software Engineering (CASE): Technology for Improving Software Development Productivity". In: *SIGMIS Database* 17.1 (Sept. 1985), pp. 35–43. ISSN: 0095-0033. DOI: 10.1145/1040694.1040698. URL: https://doi.org/10.1145/1040694.1040698.

[5] Edmund M Clarke Jr et al. *Model checking*. MIT press, 2018.

[6] Clearsy. *Atelier B*. URL: https://www.atelierb.eu/en/ (visited on 05/13/2021).

[7] Jeffery Allen Clouse. *On integrating apprentice learning and reinforcement learning*. University of Massachusetts Amherst, 1996.

[8] Alan M Davis. *Software requirements: objects, functions, and states*. Prentice-Hall, Inc., 1993.

[9] Thomas G Dietterich. "Hierarchical reinforcement learning with the MAXQ value function decomposition". In: *Journal of artificial intelligence research* 13 (2000), pp. 227–303.

[10] Kurt Driessens and Sašo Džeroski. "Integrating guidance into relational reinforcement learning". In: *Machine Learning* 57.3 (2004), pp. 271–304.

[11] Nathan Fulton and André Platzer. "Safe reinforcement learning via formal methods: Toward safe control through proof and learning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018.

[12] J. García and F. Fernández. "A comprehensive survey on safe reinforcement learning". In: 16 (Aug. 2015), pp. 1437–1480.

[13] Dragan Gaševic, Dragan Djuric, and Vladan Devedžic. *Model driven architecture and ontology development*. Springer Science & Business Media, 2006.

[14] Peter Geibel and Fritz Wysotzki. “Risk-Sensitive Reinforcement Learning Applied to Control under Constraints”. In: 24.1 (July 2005), pp. 81–108. ISSN: 1076-9757.

[15] Ian Goodfellow et al. *Deep learning*. Vol. 1. 2. MIT press Cambridge, 2016.

[16] Brent Hailpern and Peri Tarr. “Model-driven development: The good, the bad, and the ugly”. In: *IBM systems journal* 45.3 (2006), pp. 451–461.

[17] Tim Hartmann and Daniel A Sadilek. “Undoing operational steps of domain-specific modeling languages”. In: *Proceedings of the 8th OOPSLA Workshop on Domain-Specific Modeling (DSM 2008), University of Alabama at Birmingham*. 2008.

[18] Akram Idani, Yves Ledru, and German Vega. “Alliance of model-driven engineering with a proof-based formal approach”. In: *Innovations in Systems and Software Engineering* 16 (2020), pp. 289–307.

[19] Xavier Leroy. “Formal verification of a realistic compiler”. In: *Communications of the ACM* 52.7 (2009), pp. 107–115.

[20] Michael Leuschel and Michael Butler. “ProB: An Automated Analysis Toolset for the B Method”. In: *International Journal on Software Tools for Technology Transfer* 10 (2008). DOI: 10.1007/s10009-007-0063-9.

[21] Justin Le Louedec et al. “Deep learning investigation for chess player attention prediction using eye-tracking and game data”. In: *Proceedings of the 11th ACM Symposium on Eye Tracking Research & Applications*. 2019, pp. 1–9.

[22] Richard Maclin et al. “Knowledge-based support-vector regression for reinforcement learning”. In: *Reasoning, Representation, and Learning in Computer Games* (2005), p. 61.

[23] George Rupert Mason et al. “Assured reinforcement learning with formally verified abstract policies”. In: *9th International Conference on Agents and Artificial Intelligence (ICAART)*. York. 2017.

[24] Laëtitia Matignon, Guillaume Laurent, and Nadine Fort-Piat. “Reward Function and Initial Values: Better Choices for Accelerated Goal-Directed Reinforcement Learning”. In: Sept. 2006. DOI: 10.1007/11840817_87.

[25] Stefan Mitsch and André Platzer. “ModelPlex: Verified runtime validation of verified cyber-physical system models”. In: *Formal Methods in System Design* 49.1 (2016), pp. 33–74.

[26] Volodymyr Mnih et al. “Playing atari with deep reinforcement learning”. In: *arXiv preprint arXiv:1312.5602* (2013).

[27] Barak Oshri and Nishith Khandwala. *Predicting moves in chess using convolutional neural networks*. 2016.

[28] Jing Peng and Ronald J Williams. “Incremental multi-step Q-learning”. In: *Machine Learning Proceedings 1994*. Elsevier, 1994, pp. 226–232.

[29] Gavin A Rummery and Mahesan Niranjan. *On-line Q-learning using connectionist systems*. Vol. 37. University of Cambridge, Department of Engineering Cambridge, UK, 1994.

[30] Matthia Sabatelli, Marco Wiering, and Valeriu Codreanu. “Learning to Play Chess with Minimal Lookahead and Deep Value Neural Networks”. PhD thesis. Oct. 2017. DOI: 10.13140/RG.2.2.31956.71040.

[31] Matthia Sabatelli et al. “Learning to Evaluate Chess Positions with Deep Neural Networks and Limited Lookahead”. In: Jan. 2018. DOI: 10.5220/0006535502760283.

[32] Satinder Singh et al. “Convergence results for single-step on-policy reinforcement-learning algorithms”. In: *Machine learning* 38.3 (2000), pp. 287–308.

[33] Colin Snook and Michael Butler. “Verifying dynamic properties of UML models by translation to the B language and toolkit”. In: (2000).

[34] David Steinberg et al. *EMF: Eclipse Modeling Framework 2.0*. 2nd. Addison-Wesley Professional, 2009. ISBN: 0321331885.

[35] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. “Sequence to sequence learning with neural networks”. In: *arXiv preprint arXiv:1409.3215* (2014).

[36] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[37] Antony Tang et al. “What makes software design effective?” In: *Design Studies* 31.6 (2010). Special Issue Studying Professional Software Design, pp. 614–640. ISSN: 0142-694X. DOI: https://doi.org/10.1016/j.destud.2010.09.004. URL: https://www.sciencedirect.com/science/article/pii/S0142694X10000669.

[38] S. Thrun. “An Approach to Learning Mobile Robot Navigation”. In: *Robotics and Autonomous Systems* 15 (1996), pp. 301–319.

[39] Ulyana Tikhonova. “Reusable specification templates for defining dynamic semantics of DSLs”. In: *Software & Systems Modeling* 18.1 (2019), pp. 691–720.

[40] Christopher Watkins. “Learning From Delayed Rewards”. In: (Jan. 1989).